IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 26 August 2026, accepted 8 September 2026, date of publication 10 September 2026,
date of current version 16 September 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3732942

RESEARCH ARTICLE

Microsecond-Latency, Fully On-Chip Limit Order Book Prediction on an Edge FPGA: A Lightweight INT8 1D-CNN Hardware–Software Co-Design

SIYUAN LIU
School of Engineering, Chengdu College of University of Electronic Science and Technology of China, Chengdu 611731, China
e-mail: siyuan.liu@ieee.org

ABSTRACT High-frequency trading demands microsecond, deterministic inference that jitter-prone general-purpose processors cannot deliver, yet fully on-chip edge-FPGA predictors for real-time limit order book (LOB) forecasting remain underexplored. This paper presents a hardware–software co-design that maps a lightweight residual 1D-CNN onto a single standalone edge FPGA for LOB price-movement prediction. On the 144-dimensional FI-2010 dataset with 86,211 parameters, the model achieves a Macro-F1 of 0.8509 on the anchored deployment fold (CF_9). An 8-bit quantization-aware training pipeline with per-channel input weight-folding compresses the model by 3.38× for fully on-chip inference, limiting Macro-F1 degradation to only 1.78 percentage points (to 0.8331), compared with an 11.69-point drop from post-training quantization; a systematic bit-width study identifies INT8 as the Pareto-optimal operating point. Synthesized, placed and routed on a Xilinx Kintex UltraScale KU040, a fully-unrolled latency-optimized architecture reaches a deterministic 57 μs on-chip inference-core latency with zero run-to-run jitter, at a directly measured whole-board power of 3.11 W. On-board execution is bit-exact with the PyTorch INT8 reference across all 31,828 test-day samples, yielding 83.12% accuracy (95% Wilson interval: 82.71–83.53%). For single-sample inference, the standard operating mode for high-frequency trading, it achieves 5.98–10.44× lower latency and 23.8–41.6× higher energy efficiency than an NVIDIA RTX 5070 Laptop GPU (measured at 12.39 W), with the range covering both optimized and eager FP32 execution stacks. Verification artifacts, full code and model weights are available at https://github.com/siyuan-liu-research/LOB-FPGA

INDEX TERMS FPGA accelerator, hardware–software co-design, high-frequency trading, INT8 quantization-aware training, low-latency inference.

I. INTRODUCTION

In high-frequency trading (HFT), microsecond-scale latency differences directly determine the profitability of trading strategies [33]. Limit order books (LOBs) record real-time supply and demand, and short-term price forecasting from LOB snapshots is a core module for high-frequency strategies, on which deep learning has recently achieved remarkable accuracy gains [1].

However, most existing deep learning-based LOB predictors are deployed on CPU or GPU platforms, which cannot meet the stringent latency requirements of HFT: CPUs suffer unpredictable jitter from operating-system scheduling, while GPUs incur heavy single-sample (batch=1) overhead from PCIe transfers and kernel launches. Neither provides microsecond-scale deterministic latency, becoming the critical bottleneck of the HFT system.

Field-programmable gate arrays (FPGAs) have therefore been widely adopted in HFT for their deterministic latency and energy efficiency. However, most conventional FPGA designs are confined to low-level data-path tasks such as

The associate editor coordinating the review of this manuscript and approving it for publication was Ludovico Minati.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

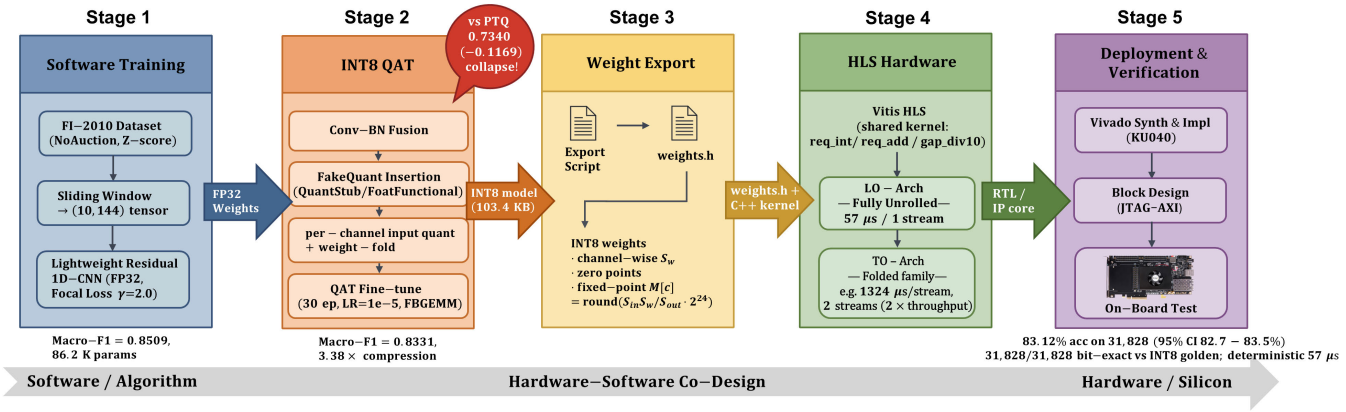


FIGURE 1. End-to-end hardware–software co-design flow from model training to on-board FPGA deployment.

network parsing and order book maintenance, embedding no deep learning module for LOB prediction. LightTrader [2], a pioneering study, couples a host FPGA with external 7 nm custom CGRA accelerators for AI-driven HFT; its reliance on custom silicon delivers strong performance but incurs prohibitive cost and long development cycles, out of reach for most small- and medium-sized trading teams. This work instead targets a lightweight end-to-end LOB prediction accelerator on a single standalone edge FPGA via high-level synthesis (HLS).

The proposed hardware–software co-design follows the flow of Fig. 1, spanning FP32 training, INT8 quantization-aware training, weight export, Vitis HLS synthesis and on-board deployment. A lightweight residual 1D-CNN is deployed on a Xilinx Kintex UltraScale KU040 FPGA with two complementary architectures: a latency-optimized architecture (LO-Arch) for single-instrument latency-critical inference, and a throughput-optimized architecture (TO-Arch) for multi-instrument monitoring. Both run entirely on-chip with cycle-accurate latency, a mode OS-dependent CPUs and bus-connected GPUs cannot provide.

A. RESEARCH GAP

Current HFT-related hardware and algorithm designs face multiple intertwined challenges, which can be grouped into three major research gaps.

1) Conventional FPGA solutions for HFT are optimized only for low-level data-path operations (network parsing, order book maintenance), while general-purpose FPGA CNN accelerators cannot meet LOB-specific requirements such as single-sample processing, low input dimensionality and ultra-low deterministic latency [3].
2) State-of-the-art deep learning models for LOB forecasting mainly focus on prediction accuracy while ignoring edge hardware constraints, making them unsuitable for resource-constrained edge FPGA devices [1].
3) Although FPGA-based HFT systems have been extensively studied, few prior works have developed fully board-verified, end-to-end LOB prediction accelerators based on HLS [2].

Taken together, lightweight, dedicated FPGA accelerators that provide deterministic on-chip inference represent a distinct and valuable research gap in the field of AI-enabled HFT.

B. CONTRIBUTIONS

This work presents four core contributions to the end-to-end design of AI-driven LOB prediction accelerators.

1) A lightweight, hardware-optimized residual 1D-CNN is designed for LOB forecasting. With only 86,211 parameters, the entire model is stored in on-chip block RAM (BRAM), which completely removes non-deterministic off-chip memory access. All accuracy results are reported under a leakage-controlled anchored walk-forward protocol with purge/embargo at every split boundary, and the test set is evaluated only once.
2) This work develops a complete INT8 quantization-aware training (QAT) pipeline integrating Conv-BN fusion, residual scale alignment, per-channel input quantization with weight-folding and Banker's rounding. This pipeline compresses the model by 3.38× while limiting the Macro-F1 loss to 1.78 percentage points, in contrast to post-training quantization which loses 11.69 points on the same model. A full-integer bit-width study, in which both weights and activations are quantized, further reveals that INT8 is the Pareto-optimal hardware bit width for LOB data, with a sharp accuracy degradation below 8 bits under post-training quantization that quantization-aware training only partly relaxes.
3) A controlled design-space exploration (DSE) of loop folding is conducted for FPGA acceleration, decoupling the two variables of folding strategy and

stream count into independent axes. Within this exploration, a dimension-aware mixed-factor folding scheme matches folding factors to the native channel dimensions to remove the extra multiplexers and pipeline bubbles introduced by uniform folding, and a folded engine can be spatially replicated so that aggregate throughput scales near-linearly with stream count at unchanged single-sample latency, as demonstrated up to two streams within the device budget. The fully-unrolled latency-optimized architecture establishes the single-stream latency lower bound at a deterministic 57 μs. All variants share a unified on-chip INT8 datapath whose predicted classes match the PyTorch INT8 reference; the deterministic latency follows from the branch-free fixed-point datapath and is corroborated on board.

4) Extensive on-board tests and cross-platform evaluations are conducted. On-board execution is bit-exact with the PyTorch INT8 reference on all tested samples, and board-level accuracy is reported with confidence intervals. In single-sample inference, the latency-optimized design is 5.98–10.44× faster and 23.8–41.6× more energy-efficient than an NVIDIA RTX 5070 Laptop GPU whose steady-state power is physically measured, the range spanning an optimized ONNX Runtime CUDA engine and an eager-PyTorch baseline at matched FP32 precision. The primary contribution is a system one: a board-verified, fully on-chip INT8 LOB inference datapath on a single standalone edge-class FPGA that uses no external coarse-grained reconfigurable accelerator and no off-chip weight storage, in contrast to CGRA-coupled systems such as LightTrader [2].

The proposed accelerator is an on-chip inference core that consumes a precomputed 144-dimensional feature window; feed decoding, order-book reconstruction, feature computation, normalization and order generation are performed upstream and are outside the scope of the reported latency. This work therefore targets the inference stage of an HFT pipeline.

C. PAPER ORGANIZATION

Section II reviews related work and Section III the preliminaries. Section IV presents the 1D-CNN and INT8 QAT pipeline, and Section V the Vitis HLS micro-architecture and the two accelerator designs. Section VI reports the experimental results, Section VII discusses deployment, scalability and limitations, and Section VIII concludes.

II. RELATED WORK

A. DEEP LEARNING FOR LOB PREDICTION

Short-term price forecasting based on LOBs is a core task in high-frequency trading. Traditional models rely on hand-engineered features including spreads, averages and volatilities calculated by fixed formulas, yet they fail to capture the nonlinear dynamics of LOB data. Early work introduced convolutional neural networks to LOB-based stock price forecasting [34]. As a pioneering end-to-end framework, DeepLOB integrates convolutional layers, Inception modules and long short-term memory (LSTM) layers [1]. Other lines of work include bilinear temporal networks such as the Temporal Attention-Augmented Bilinear Network [43], and more recent Transformer-based predictors such as TLOB, which introduces dual attention over the time and feature axes [44], and the limit order book transformer LiT [56]. A recent benchmark study further shows that reported accuracies on FI-2010 depend strongly on the evaluation protocol, so numbers obtained under different splits are not directly comparable [45]; all baselines are therefore re-run under one identical protocol for a fair comparison (Section VI-A). From the perspective of market microstructure, recent work shows that stock spreads and liquidity greatly affect prediction accuracy, further strengthening the theoretical basis of LOB forecasting [5]. Lightweight LOB predictors have also been explored for mobile and edge computing platforms [46], although not with the microsecond deterministic latency targeted here.

Several studies have implemented 1D-CNNs on FPGAs for foreign exchange trend prediction, yet they are not tailored to the inherent properties of LOB data—tick-by-tick streaming and microsecond deterministic latency—and thus cannot support real-time high-frequency LOB forecasting [6]. More broadly, deep learning has continued to advance rapidly across other high-throughput sensing and detection domains [47], [48], [49], reflecting a wider push toward efficient inference on data-intensive workloads.

B. FPGA INFRASTRUCTURE FOR HFT

Existing studies have implemented full-stack, low-latency HFT systems using HLS, covering data-path tasks such as network parsing and order book processing [7], [29]. The most closely related work is LightTrader [2], which integrates deep learning into HFT via an external CGRA accelerator linked by high-bandwidth interconnects. It thus differs from this work in three respects: it runs inference off-chip rather than entirely on a single standalone KU040; it relies on a heterogeneous design instead of an all-in-one HLS workflow; and it targets data-center-class rather than edge-class FPGAs requiring full on-chip deployment at microsecond deterministic latency.

Prior works on FPGAs have realized low-latency order book maintenance [8], hardware acceleration for traditional technical indicators (MACD and RSI) [9], and real-time order book construction and snapshot generation [10], as well as reusable low-latency FPGA IP libraries for financial protocol parsing [39]. However, all these solutions only focus on basic low-level data processing and lack integrated deep learning-based LOB prediction, which cannot support end-to-end intelligent trading decisions. Beyond FPGAs, programmable network switches have also been leveraged to

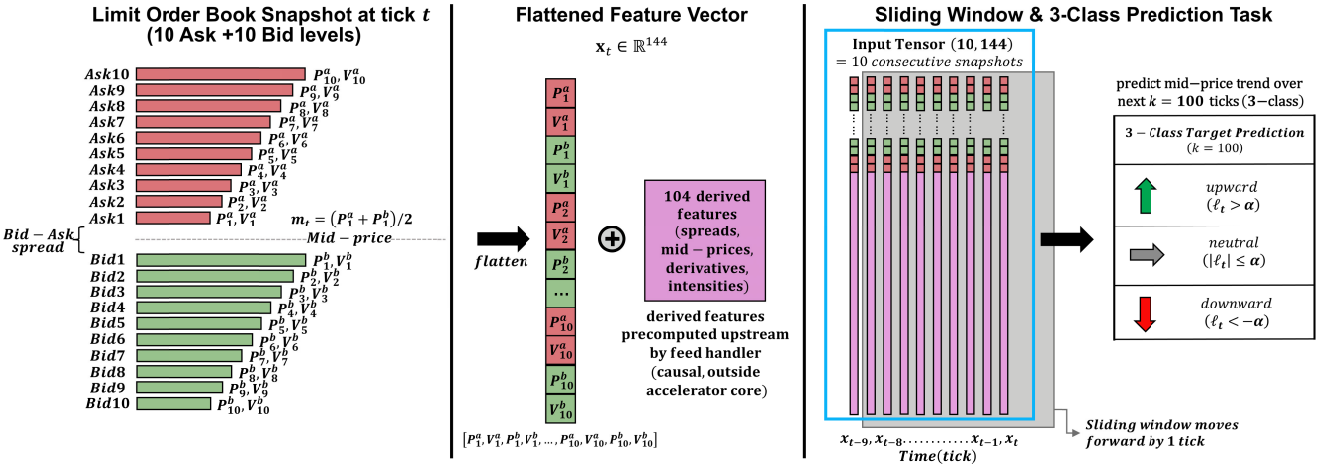


FIGURE 2. LOB structure and sliding-window construction of the (10, 144) input tensor.

accelerate trading machine learning directly in the network data path [57], further illustrating the drive toward in-line, low-latency inference for HFT.

C. FPGA-BASED CNN INFERENCE ACCELERATION

Leveraging inherent parallelism, FPGAs are widely adopted for lightweight CNN inference [30]. The NN2FPGA framework maps residual networks onto FPGAs via binary integer programming with prominent throughput gains [11], and lightweight 1D-CNN accelerators for edge devices have been applied to real-time stream processing and ECG classification [12], [13], confirming the fit between 1D-CNN models and FPGA pipelines. The drive toward efficient neural inference under tight throughput and latency budgets is likewise evident across other large-scale sensing and detection tasks [50], [51], [52].

Nevertheless, most existing accelerators target general speech and sensor data, or large-scene visual detection and recognition pipelines [53], [54], [55], not financial LOB data with its single-sample streaming input and deterministic-latency requirements. Existing co-design solutions run lightweight CNNs on edge FPGAs at millisecond latency and sub-watt power [18], but still cannot meet the microsecond-scale demands of HFT [26].

D. QUANTIZATION AND SOFTWARE-HARDWARE CO-DESIGN

Quantization reduces storage and computation for edge AI: improved Banker's rounding lowers FPGA fixed-point error [14], QAT bounds full-integer accuracy loss [15], [37], and FPGAs can trade off floating- and fixed-point modes for financial computing [16]. Co-design jointly optimizes model structure, numerical format and hardware dataflow [31], with HLS easing FPGA development [18], compression-aware frameworks deploying lightweight CNNs [17], and DSE trading off resource, latency and throughput [19]. Yet existing quantization schemes neglect the precision sensitivity of LOB data, and off-the-shelf co-design schemes meet neither microsecond-scale deterministic latency nor complete end-to-end board-level verification; no INT8 pipeline exists that is designed for financial time series and tightly coupled to FPGA hardware.

III. PRELIMINARIES

A. LOB AND DATA SPECIFICATION

The LOB records real-time prices and volumes of all pending limit orders, reflecting instantaneous market supply and demand [21]. Each snapshot contains 10 price levels on both the ask and bid sides, forming a 40-dimensional raw price–volume vector x_t^{raw}:

$$x_t^{\text{raw}} = \left[P_1^a, V_1^a, P_1^b, V_1^b, \ldots, P_{10}^a, V_{10}^a, P_{10}^b, V_{10}^b\right]^\top \in \mathbb{R}^{40} \tag{1}$$

P_i^a and V_i^a denote the i-th ask price and ask volume, while P_i^b and V_i^b stand for the i-th bid price and bid volume. On top of these raw levels, the FI-2010 benchmark provides a standardized 144-dimensional feature vector $x_t \in \mathbb{R}^{144}$ per timestep, comprising the 40 raw dimensions together with 104 handcrafted features such as bid–ask spreads, mid-prices, price and volume derivatives, and accumulated trading intensities [21]. Every one of these features depends only on information available up to time t, so the vector is strictly causal and introduces no look-ahead bias. This work uses the full 144-dimensional vector as the network input; the derived features are produced by the upstream market-data feed handler as part of causal preprocessing, in the same manner as the Z-score normalization of Section III-C, and therefore lie outside the accelerator core whose latency is characterized in Section V. The impact of using the 40 raw dimensions versus the full 144-dimensional set is quantified via ablation in Section IV-A. The LOB structure and the

sliding-window construction of the (10, 144) input tensor with the three-class label are illustrated in Fig. 2.

LOB data update tick-by-tick at the microsecond scale, imposing strict inference-latency and timing-determinism requirements that define the core hardware-acceleration demands for HFT [7].

B. LOB PRICE PREDICTION TASK

This work defines LOB-based short-term price forecasting as a three-class problem: predicting the mid-price trend (upward, neutral or downward) over the next 100 ticks [1]. Input samples are generated by the (10, 144) sliding window defined in Section III-A, and all data are split strictly in chronological order to eliminate look-ahead bias [1], [21].

Label assignment complies with the official FI-2010 benchmark [21]. The mid-price is first computed from the best bid and best ask prices [20]:

$$m_t = \frac{P_1^a + P_1^b}{2} \tag{2}$$

where P_1^a and P_1^b stand for the best ask price and best bid price at time t.

Based on the FI-2010 labeling rule with a prediction horizon $k = 100$ and a predefined price change threshold α, the relative mid-price variation is converted into three categories:

$$m_+(t) = \frac{1}{k}\sum_{i=1}^{k} m_{t+i}, \qquad \ell_t = \frac{m_+(t) - m_t}{m_t} \tag{3}$$

$$y_t = \begin{cases} 0 \text{ (upward)} & \ell_t > \alpha \\ 1 \text{ (neutral)} & |\ell_t| \le \alpha \\ 2 \text{ (downward)} & \ell_t < -\alpha \end{cases} \tag{4}$$

In the above formulas, $m_+(t)$ is the average mid-price over the future k ticks, and ℓ_t denotes the relative return of mid-price [1], [21]. Ultimately, a deterministic mapping from the input tensor $\mathbb{R}^{10\times144}$ to the label set $\{0, 1, 2\}$ is constructed for prediction.

C. Z-SCORE NORMALIZATION

Raw LOB features differ greatly in magnitude: price levels are large and slow-varying, whereas order volumes are small and bursty. Feeding such unnormalized data into the network impairs training stability and INT8 inference accuracy. Z-score normalization $\tilde{x}_{t,j} = (x_{t,j} - \mu_j)/\sigma_j$ is therefore applied to standardize all 144 features to zero mean and unit variance, where μ_j and σ_j are the per-feature mean and standard deviation; following benchmark conventions, they are computed from prior trading days, which is causal and avoids look-ahead bias [21].

D. 1D CONVOLUTIONAL NEURAL NETWORK (1D-CNN)

One-dimensional CNNs (1D-CNNs) extract local temporal features via sliding kernels and, lacking recurrent dependencies, map fully to spatially parallel FPGA pipelines, making them well-suited for low-latency HFT inference [3], [12], [22], [23], [36]. The 1D convolution with stride $s = 1$ and padding p is defined as:

$$y[c_{\text{out}}, t] = \sum_{c_{\text{in}}=0}^{C_{in}-1}\sum_{j=0}^{K-1} w[c_{\text{out}}, c_{\text{in}}, j]\; x[c_{\text{in}},\; t+j-p] + b[c_{\text{out}}] \tag{5}$$

where C_{in}, C_{out}, T and K denote the input/output channel counts, temporal length and kernel size. A same-padding scheme ($K = 3$, $s = 1$, $p = 1$) preserves the temporal dimension for the fixed-latency FPGA pipeline, and ReLU is used as the activation since it maps to simple comparators with no DSP cost.

E. RESIDUAL BLOCK AND CONV-BN FUSION

Residual blocks address gradient vanishing via shortcut connections, $\mathbf{y} = \text{ReLU}(\mathbf{x} + \mathcal{F}(\mathbf{x}))$, where the residual path $\mathcal{F}(\mathbf{x})$ contains two convolutions, batch normalization (BN) and ReLU. Being mostly additions, they incur very low hardware overhead and are FPGA-friendly [11]. At inference BN reduces to a fixed affine transform, so Conv-BN fusion embeds the BN parameters into the convolution weights and biases, eliminating standalone BN and cutting on-chip computation [11], [15], [24].

F. GLOBAL AVERAGE POOLING (GAP)

GAP compresses the temporal dimension ($T = 10$) by channel-wise averaging. Being parameter-free, it replaces the large fully-connected weight matrix and removes the associated BRAM bottleneck on FPGAs [22].

G. INT8 QUANTIZATION AND BANKER'S ROUNDING

INT8 quantization maps FP32 floating-point data to 8-bit integers. This technique brings a 4× compression ratio and faster computation, while removing the overhead of floating-point arithmetic on FPGAs [4], [25]. In practical deployment, activations usually adopt asymmetric quantization and weights use channel-wise symmetric quantization [25]. The full quantization workflow of the present design is described in Section IV-C.

Activation Quantization:

$$x_{\text{int}} = \text{clip}\left(\text{round}\left(\frac{x}{S_x} + Z_x\right), 0, 255\right) \tag{6}$$

Weight Quantization:

$$w_{\text{int}}^{(c)} = \text{clip}\left(\text{round}\left(\frac{w^{(c)}}{S_w^{(c)}}\right), -127, 127\right) \tag{7}$$

Here S_x and $S_w^{(c)}$ are quantization scale factors, and Z_x is the activation zero point; quantized activations lie in $[0, 255]$ and weights in $[-127, 127]$ [25].

To suppress cumulative quantization errors, Banker's rounding (round half to even) is employed, matching the

TABLE 1. Feature-dimension ablation on FI-2010 (NoAuction, anchored fold CF_9).

Feature Set	Params	Macro-F1	On-Chip Preprocessing
144-dim (adopted)	86.2 K	0.8509	None
40-dim (raw only)	66.2 K	0.7227	None

PyTorch default [14]. Implemented with basic combinational logic, it preserves accuracy at negligible resource cost [4], [14].

H. FPGA HLS FUNDAMENTALS

HLS converts C/C++ into RTL and exposes pragmas for DSE [17]. The proposed accelerator adopts an OS-less, pure-streaming architecture with an INT8 datapath and 32-bit accumulation to avoid overflow; its dedicated microarchitecture for the residual 1D-CNN is elaborated in Section V.

IV. SOFTWARE PIPELINE: LIGHTWEIGHT 1D-CNN AND INT8 QUANTIZATION-AWARE TRAINING

This section presents the software pipeline that produces bit-accurate INT8 models for the FPGA accelerator in Section V, satisfying the three HFT constraints in Section I: on-chip weight storage, single-sample streaming and deterministic microsecond latency.

All experimental results in this section are fully reproducible with the configurations in Table 4 (random seed 3407 for Python, NumPy, PyTorch and CUDA).

A. DATASET CONSTRUCTION AND RAW-FEATURE SELECTION

1) FEATURE SET AND THE COST OF RAW-ONLY INPUTS

The 144-dimensional FI-2010 feature vector (40 raw dimensions plus 104 handcrafted features) defined in Section III-A is strictly causal and produced by the upstream feed handler alongside Z-score normalization [21], so it lies outside the on-chip accelerator core and does not enter its latency budget.

A feature-dimension ablation is conducted under the leakage-controlled protocol of Section IV-A4, and the results are summarized in Table 1. Using the full 144-dimensional input, the network reaches a Macro-F1 of 0.8509, whereas restricting the input to the 40 raw dimensions drops the Macro-F1 to 0.7227, a loss of 12.8 percentage points. This large gap indicates that, under the leakage-controlled walk-forward protocol, the handcrafted micro-structural features carry substantial predictive information that a compact 1D-CNN cannot fully recover from raw levels alone within ten timesteps. Since these features are precomputed upstream and only widen the first convolution layer on chip (from 40 to 144 input channels, as detailed in Section V), the full 144-dimensional set is adopted as the network input.

The 40-dimensional variant is retained only as an ablation reference; the accuracy penalty of discarding the micro-structural features is not justified for the target application.

2) SLIDING WINDOW AND LABEL DEFINITION

Following the sliding window mechanism and three-class labeling rule introduced in Section III-B, temporal samples with the dimension of $(10, 144)$ are constructed. A window of $T = 10$ consecutive snapshots follows the standard FI-2010 setup adopted by prior LOB predictors [1], [21], and keeps the input tensor small enough for a fully on-chip, fixed-latency datapath. Class labels are directly taken from the official pre-defined annotations of the FI-2010 dataset for a prediction horizon of $k = 100$ [21]. This horizon achieves a balanced distribution among three classes, while shorter horizons ($k = 10, 20$) lead to an over-represented neutral class and are easily affected by market microstructure noise. All samples are sorted strictly in chronological order without random shuffling, which thoroughly eliminates look-ahead bias.

3) NORMALIZATION STATISTICS AND QUANTIZATION RANGE

Z-score normalization is applied to all inputs following Section III-C. An empirical check confirms that 99.2% of normalized features lie within $[-3, 3]$, verifying compatibility with symmetric INT8 quantization and suppressing outlier-induced clipping errors [25].

4) LEAKAGE-CONTROLLED ANCHORED WALK-FORWARD PARTITIONING

The FI-2010 NoAuction subset is organized as nine anchored (cumulative) day folds. Rather than concatenating folds, which would repeat early trading days and overlap training and test periods, this work follows the official anchored walk-forward protocol: for fold CF_k, the training pool is the single cumulative file Train_CF$_k$ (days 1 to k) and the test set is the matching held-out file Test_CF$_k$ (day $k+1$), which are strictly disjoint in time. The main results are reported on fold CF_9 (train: days 1–9; test: day 10).

Because each label depends on the mid-price over the next $k = 100$ ticks, a simple chronological split still leaks information across split boundaries. A purge/embargo gap of $T + k = 110$ ticks is therefore removed at both the train/validation and validation/test boundaries, and sliding windows are never constructed across day boundaries. The validation set is the last 10% of the training pool in chronological order, and is used for all model-selection decisions, including architecture, kernel size, quantization configuration and bit width; the test set is evaluated once. On fold CF_9 the training pool contains 325,069 samples and the test set contains 31,828 samples; the training class distribution is upward 39.4%, neutral 22.3% and downward 38.3%.

TABLE 2. Kernel-size ablation (144-dim input, anchored fold CF_9).

Kernel Size	Parameters	Macro-F1
3	86.2 K	0.8509
5	137.4 K	0.8407
7	188.6 K	0.8450

B. HARDWARE-FRIENDLY RESIDUAL 1D-CNN ARCHITECTURE

1) ARCHITECTURE OVERVIEW

A feedforward residual 1D-CNN is designed for this task, and its overall structure, shown in Fig. 3, is described as follows: an input convolution block (Conv1: 144 → 64 channels), one residual block (ResBlock: 64 → 64), a second convolution block (Conv3: 64 → 128), a global average pooling layer for temporal dimension compression, and a two-layer fully connected classifier that outputs three prediction logits (FC1: 128 → 64, FC2: 64 → 3). All convolutional layers adopt a unified configuration with kernel size $K = 3$ and padding size 1.

The temporal dimension remains unchanged throughout feature extraction, which is a key prerequisite for the fully-unrolled FPGA design with fixed latency presented in Section V [10]. Before feeding data into the first convolution layer, the input tensor is transposed from $(B, T, C) = (B, 10, 144)$ to $(B, 144, 10)$, so that convolutional kernels can slide along the temporal axis, as illustrated in Eq. (5).

The kernel size $K = 3$ is selected because it is optimal on both accuracy and hardware cost, rather than representing a compromise between them. As shown in Table 2, under the leakage-controlled protocol the larger kernels $K = 5$ and $K = 7$ do not improve accuracy: their Macro-F1 scores of 0.8407 and 0.8450 are both below the 0.8509 of $K = 3$, while inflating the parameter count by 1.6× and 2.2×, respectively. The small difference between $K = 5$ and $K = 7$ is within run-to-run variation and does not reverse this ordering; the essential point is that no larger kernel exceeds $K = 3$. A kernel of $K = 3$ therefore attains the highest Macro-F1 with the fewest parameters, and its 86.2 K INT8 parameters fit comfortably within the 2.64 MB on-chip BRAM of the Kintex KU040.

Since the larger kernels bring no accuracy benefit while consuming more parameters, DSP and LUT resources in the fully-unrolled datapath of Section V-E, $K = 3$ is adopted for all convolutional layers as the choice that is simultaneously the most accurate and the most hardware-efficient.

2) HARDWARE-FRIENDLY DESIGN CHOICES

All network modules are optimized for the spatial pipelined FPGA datapath of Section V [10].

- **Residual Connection:** In the quantized inference flow, residual branches only perform integer addition without consuming any DSP units.

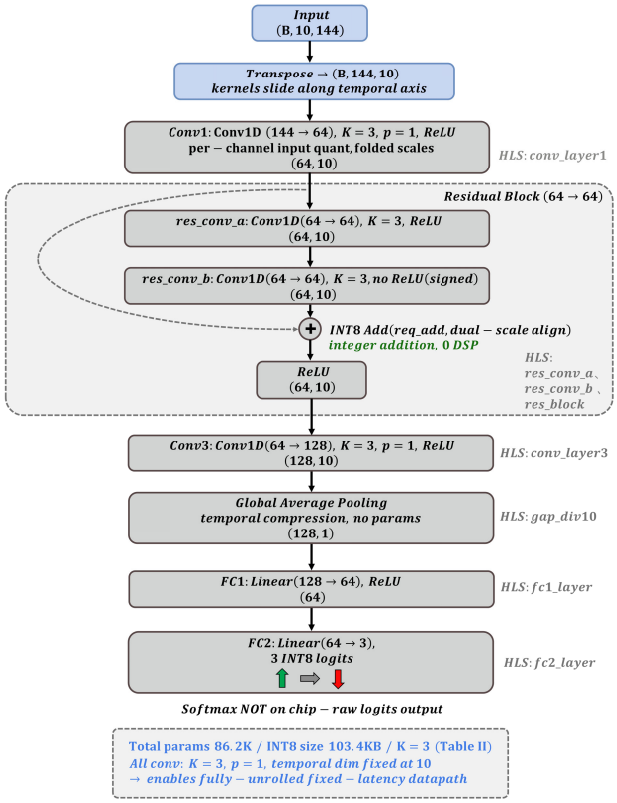


FIGURE 3. Architecture of the proposed lightweight residual 1D-CNN.

- **Global Average Pooling (GAP):** GAP replaces tensor flattening and fully connected layers, cutting the classifier's parameter overhead and eliminating the BRAM bottleneck on FPGAs.
- **ReLU Activation:** ReLU maps to simple signed comparators and multiplexers, requiring no DSPs or floating-point units (FPUs).
- **Logits-Only Output:** Softmax is not deployed on the FPGA; the accelerator outputs only raw INT8 logits, avoiding on-chip exponentials [25].
- **Classifier Dropout ($p = 0.3$):** Dropout is active only during training, adding no inference-time hardware overhead.

3) LOSS FUNCTION AND DESIGN-CHOICE ABLATION

In the fold CF_9 training pool, the neutral class occupies 22.3% of all samples while the upward and downward classes account for 39.4% and 38.3%. Although this imbalance is moderate rather than extreme, the hard cases concentrate near the neutral-versus-directional boundaries; Focal Loss is chosen less for the class-count ratio than for down-weighting the many easy samples so training focuses on these boundary cases, where directional recall is won or lost. The model is trained with Focal Loss without class-balancing weights ($\alpha_t = 1$) and a focusing factor of $\gamma = 2.0$, formulated as:

$$L_{\mathrm{FL}}(p_t) = -(1 - p_t)^{\gamma} \log(p_t) \tag{8}$$

TABLE 3. Residual and focal-loss ablation (144-dim, fold CF_9).

Configuration	Val Macro-F1 (5 seeds)	FP32 test	INT8 test	Quant. loss
Plain CNN + CE	0.8562 ± 0.0027	0.8536	0.7691	0.0845
Residual CNN + CE	0.8515 ± 0.0046	0.8471	0.8409	0.0062
Residual CNN + Focal (deployed)	0.8510 ± 0.0033	0.8509	0.8331	0.0178

where p_t is the predicted probability of the ground-truth class [27]. Focal Loss acts only during training and adds no FPGA inference overhead.

To assess whether the residual connection and Focal Loss actually improve accuracy, a controlled ablation is run over three configurations that share an identical layer shape and differ only in the shortcut and the loss: a plain CNN with cross-entropy, a residual CNN with cross-entropy, and the deployed residual CNN with Focal Loss. Because a single seed is not sufficient to separate a genuine effect from training noise, each configuration is trained under five random seeds {3407, 42, 123, 2024, 7}, and to respect the test-set discipline of Section IV-A4 model selection is decided on the validation set. As summarized in the first two columns of Table 3, the three configurations obtain validation Macro-F1 of 0.8562 ± 0.0027, 0.8515 ± 0.0046 and 0.8510 ± 0.0033; the total spread across all three means is only 0.52 percentage points and their $\pm\sigma$ intervals overlap, so the differences are not statistically significant, and the single-seed ordering is dominated by seed variance.

Because the three configurations are statistically indistinguishable in FP32, the deciding criterion is behaviour under the INT8 deployment the accelerator actually runs. The last three columns of Table 3 report, for the deployed seed, the FP32 and INT8 test Macro-F1 after the identical QAT and fixed-point export flow of Section IV-C. The plain-CNN and residual-CE rows report the independent per-configuration QAT→INT8 rerun, whereas for the deployed residual+Focal row the INT8 value is the exact deployed fixed-point export (0.8331, as used throughout the paper); the corresponding independent rerun gives 0.8314, a $\approx$ 0.2-point run-to-run difference, so the qualitative ordering across configurations is unchanged. Here the configurations separate sharply: the plain CNN has the highest FP32 test Macro-F1 (0.8536) yet collapses by 8.4 points to 0.7691 under full-integer quantization, whereas both residual variants stay within two points of their FP32 scores (the deployed residual CNN with Focal Loss drops only 1.78 points to 0.8331). The residual shortcut—a single integer-addition path that costs negligible on-chip resource and leaves the layer shape, latency and DSP/LUT footprint essentially unchanged—thus provides the quantization robustness that the plain topology lacks, so at INT8 the deployed model outperforms the nominally more accurate plain CNN by about six points. This is why the plain CNN, despite winning in FP32, is not deployed and is not carried to hardware: it is eliminated already at the software INT8 stage, and synthesizing it would only reproduce a datapath that is bit-exact with a weaker software model. Between the two residual variants, Focal Loss is preferred because at statistically equivalent aggregate Macro-F1 it rebalances the operating point across the directional classes: on the deployed seed, relative to cross-entropy it trades a portion of upward-class recall (from 0.836 down to 0.807) while leaving neutral and downward recall essentially unchanged (about 0.83 and 0.85), down-weighting the many easy samples so that training concentrates on the harder neutral-versus-directional boundary cases. This yields the more balanced precision–recall trade-off across the three classes that a directional trading signal benefits from, at no cost in aggregate Macro-F1. All subsequent hardware evaluations use this fixed configuration, whose board-level bit-exactness with the PyTorch INT8 reference is established in Section VI-B.

TABLE 4. Training configurations for the FP32 baseline model.

Hyperparameter	Value
Optimizer	Adam
Initial learning rate	1×10^{-3}
Learning rate scheduler	Cosine annealing ($T_{\max} = 100$, $\eta_{\min} = 1 \times 10^{-5}$)
Batch size	512
Maximum number of epochs	100
Early stopping patience	10
Validation set ratio	0.1
Loss function	Focal Loss ($\gamma = 2.0$)
Classifier Dropout	0.3
Random seed	3407 (Python, NumPy, PyTorch, CUDA)
Training hardware	NVIDIA RTX 5070 Laptop GPU
Framework	PyTorch 2.11.0 (CUDA 12.8)

4) TRAINING CONFIGURATION

All training for the FP32 baseline model is performed with the hyperparameters summarized in Table 4. All settings are fixed strictly to ensure full reproducibility of experimental results.

With the above configurations, the FP32 baseline model reaches a test accuracy of 84.96% and an overall Macro-F1 score of 0.8509 on the held-out test day of fold CF_9. The F1 scores for upward, neutral and downward trends are 0.8508, 0.8667 and 0.8351, respectively.

5) ARCHITECTURE ABLATION

To compare the 1D-CNN against other temporal models under matched conditions, Multilayer Perceptron (MLP) and

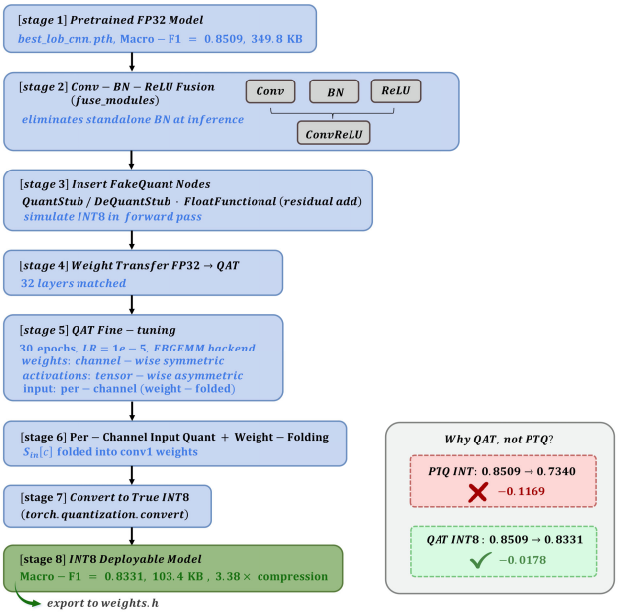


FIGURE 4. Overview of the proposed INT8 quantization-aware training pipeline.

Long Short-Term Memory (LSTM) baselines are trained with exactly the same hyperparameters, loss function and chronological data partitioning, with the network structure as the only variable [1]; the corresponding Macro-F1 values also appear in the comparable zone of Table 9.

The MLP enables full spatial parallelism but discards the temporal ordering of the window, and despite the largest parameter budget (193.3 K) reaches only 0.8243 [1]. The LSTM (87.2 K parameters) scores lowest (0.7986): it overfits the short ten-step window, and its serial cross-step dependency prevents a fully pipelined FPGA datapath, incompatible with deterministic ultra-low latency [10]. Attention-based forecasters such as Transformers [42] likewise rely on Softmax and large matrix operations costly on resource-constrained FPGAs; a Transformer baseline is evaluated in Section VI-A. Among these temporal-model baselines the residual 1D-CNN attains the highest Macro-F1 (0.8509) with the fewest parameters (86.2 K) while retaining full spatial parallelism, and as a pure feed-forward network strictly maps to a fully parallel datapath—the strongest choice under the practical joint accuracy–hardware constraints, and the basis for Section V. Recurrent designs such as DeepLOB reach a marginally higher Macro-F1 (Section VI-A) but do not map to a deterministic fully-unrolled pipeline.

C. INT8 QUANTIZATION-AWARE TRAINING PIPELINE

1) MOTIVATION: PTQ FAILURE ON LOB DATA

The 144-dimensional LOB feature vector is highly heterogeneous: price levels are large and slow-varying, order volumes are small and bursty, and the handcrafted spreads, mid-prices, derivatives and accumulated intensities span widely different numerical ranges. When post-training quantization (PTQ) is directly applied to the FP32 baseline, the Macro-F1 score declines drastically from 0.8509 to 0.7340, an absolute loss of 11.69 percentage points. The dominant cause is the input stage: forcing all 144 heterogeneous channels through a single per-tensor input quantizer lets a few wide-range channels stretch the scale, so the remaining channels are quantized too coarsely.

To tackle this challenge, QAT is adopted so that the network learns to compensate for quantization noise during fine-tuning [15], [25], and the input stage is additionally handled with per-channel input quantization folded into the first convolution, as detailed below. This restores most of the lost accuracy and limits the final INT8 loss to 1.78 percentage points.

2) CORE COMPONENTS OF THE QAT PIPELINE

The presented QAT workflow, illustrated in Fig. 4, incorporates four mature optimization techniques, all compatible with the FBGEMM INT8 backend supported by the PyTorch quantization framework [25]. Together, these techniques ensure that the quantized model and the corresponding FPGA accelerator produce identical predicted classes described in Section V [25].

- **Per-channel input quantization with weight-folding:** Each of the 144 input channels receives its own 8-bit scale $s_{\text{in}}[c]$ and zero point, so heterogeneous features no longer share one dynamic range. As a per-channel input scale cannot be captured by the first convolution's single requantization multiplier, each scale is folded into the first-layer weights, $W_{\text{eff}}[o, c, \cdot] = s_{\text{in}}[c]\, W[o, c, \cdot]$, then symmetrically re-quantized per output channel. The convolution thus keeps its structure while consuming per-channel integer inputs, storing only the 144 input zero points as an array. This step alone recovers most of the accuracy lost to per-tensor input quantization.
- **Mixed-granularity quantization:** Weights of convolution and fully connected layers use channel-wise symmetric quantization, and internal activations use tensor-wise asymmetric quantization [25], following Eq. (6) and Eq. (7). This maximizes the dynamic range of per-channel weight distributions and exploits the non-negative ReLU outputs.
- **Conv-BN-ReLU ternary fusion:** Following the Conv-BN fusion of Section III-E, all Conv-BN-ReLU sequences are fused into one quantized convolution operator before INT8 quantization via PyTorch's fuse_modules, eliminating separate batch-normalization at inference and cutting on-chip memory access [15].
- **Residual branch scale alignment:** The shortcut and residual branches hold independent INT8 scales after quantization, so direct integer addition would overflow unpredictably at the addition node. The quantized FloatFunctional operator therefore unifies the two scales before summation, keeping the residual addition in pure

TABLE 5. Performance comparison of quantization strategies (144-dim, fold CF_9).

Precision	Macro-F1	Absolute Macro-F1 Loss	Model Size	Compression Ratio
FP32 (baseline)	0.8509	—	349.8 KB	1.00× (baseline)
PTQ INT8	0.7340	-0.1169	103.4 KB	3.38×
QAT INT8	0.8331	-0.0178	103.4 KB	3.38×

integer arithmetic and bit-exact with the PyTorch INT8 reference at this node.

- **Banker's rounding:** All quantization steps use round-half-to-even rounding, which yields lower accumulated bias than round-half-up during long INT8 multiply-accumulate chains [14], as detailed in Section III-G.

Starting from the FP32 champion weights, QAT fine-tuning uses a learning rate of 1×10^{-5} for 30 epochs; model selection uses a bit-exact integer forward pass on the validation set, and all other hyperparameters follow Table 4.

3) QUANTIZATION RESULTS

The performance of the FP32 baseline, conventional PTQ and the proposed QAT pipeline is compared on the held-out test set of fold CF_9, as summarized in Table 5 [21].

The presented QAT scheme compresses the model by 3.38× to 103.4 KB (on-disk `state_dict` size, measured identically for both precisions) while restricting the absolute Macro-F1 loss to 1.78 percentage points, and it outperforms PTQ by 9.91 percentage points on the same model. This 103.4 KB exceeds the bare 86,211 INT8 weights (≈ 84 KB) because it also stores the INT32 biases, the per-channel requantization multipliers and the per-channel input scales/zero points. The compressed model fits entirely within the on-chip BRAM of the Kintex UltraScale KU040, which is the key prerequisite for the microsecond-scale deterministic latency [8], [10], [25].

4) ACROSS-FOLD GENERALIZATION

To characterize stability beyond the single deployment fold, the FP32 baseline and the deployed QAT INT8 model are evaluated on all nine anchored walk-forward folds under the identical leakage-controlled protocol (Table 6). Macro-F1 broadly increases with the training-window size, from 0.6154 (FP32) on CF_1, which trains on a single day, to about 0.85 on the later folds, and the last three folds (CF_7–CF_9) converge to within about 0.004 of one another; selecting CF_9 as the deployment fold therefore reflects the most data-rich, converged operating point rather than a cherry-picked result. The across-fold mean Macro-F1 is 0.7752 ± 0.0824 (FP32) and 0.7399 ± 0.0812 (INT8); the large standard deviation is expected under anchored walk-forward, where early folds are data-starved, and confirms that a single fold does not represent a full-benchmark average. The INT8 quantization

TABLE 6. Across-fold generalization on FI-2010 (144-dim, anchored walk-forward).

Fold	FP32 Macro-F1	INT8 Macro-F1	Quant. loss
CF_1	0.6154	0.5918	0.0236
CF_2	0.6847	0.6568	0.0279
CF_3	0.7389	0.7130	0.0259
CF_4	0.7717	0.7540	0.0177
CF_5	0.8029	0.7037	0.0992
CF_6	0.8100	0.7767	0.0333
CF_7	0.8492	0.8011	0.0481
CF_8	0.8532	0.8289	0.0243
CF_9	0.8509	0.8331	0.0178
Mean±Std	0.7752 ± 0.0824	0.7399 ± 0.0812	0.0353

loss is small and stable, about two to three points on most folds (mean 0.0353), the clear exception being CF_5, where the INT8 model drops by about 9.9 points while its FP32 counterpart stays on the normal trend. Because this gap is confined to the quantized model—its INT8 validation Macro-F1 collapses in lockstep, to 0.612 versus 0.731 and 0.748 on the neighbouring CF_4 and CF_6 folds, so the effect is present on the validation split too and is not a test-set coincidence or a cherry-picking artifact—it points to the QAT calibration on this fold being unusually sensitive to the fold's activation statistics rather than to a data-distribution shift, which would degrade the FP32 model as well; the precise mechanism is left to future work. The headline results of this paper are thus reported on CF_9 as one final-day deployment fold, not as a nine-fold benchmark average. For CF_9 the across-fold sweep re-runs the QAT→INT8 pipeline independently per fold and lists an INT8 Macro-F1 of 0.8300 (and the model-selection ablation, run under the same independent-rerun setting, lists 0.8314), whereas the deployed fixed-point parameters used throughout the paper give 0.8331; these differ only by run-to-run rounding of the QAT fine-tune (≈ 0.3 point). The reported INT8 mean of 0.7399 uses the deployed CF_9 value (0.8331) for consistency with the headline number and Table 5; substituting the independent-sweep CF_9 value (0.8300) changes the nine-fold mean only marginally, to 0.7396. For compactness Table 6 lists per-fold Macro-F1 and quantization drop; the corresponding per-fold accuracy (across-fold mean 0.7823 ± 0.0745 FP32, 0.7505 ± 0.0710 INT8) and per-class recall/F1 with confusion matrices for all nine folds are provided in the Supplementary Material (9_across_fold/).

5) WEIGHT EXPORT TO THE HLS PIPELINE

Quantized integer weights, channel-wise scaling factors and tensor-wise activation zero points are exported to a C++ header file named `weights.h`. This file is directly called by the Vitis HLS microarchitecture introduced in Section V [10].

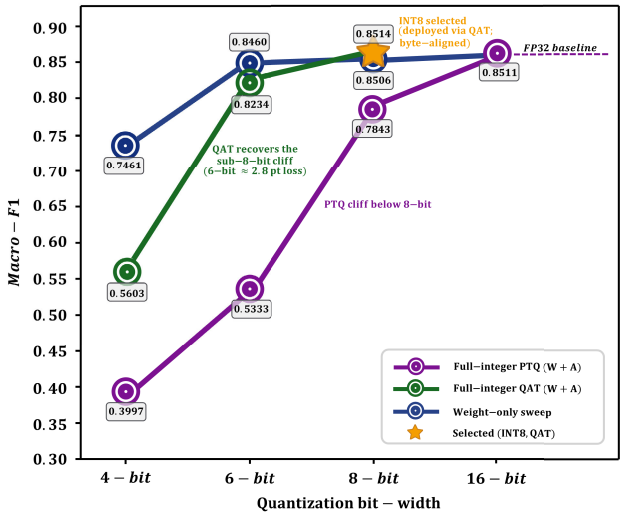


FIGURE 5. Bit-width sensitivity analysis of the proposed 1D-CNN, showing the full-integer (weights and activations) sweep against the weight-only sweep.

For each network layer the export script pre-calculates a fixed-point quantization multiplier $M = \text{round}\left(\frac{S_{\text{in}} \cdot S_{\text{w}}}{S_{\text{out}}} \cdot 2^{24}\right)$, where S_{in}, S_{w} and S_{out} are the input-activation, channel-wise weight and output-activation scales; the per-channel form $M[c]$ used on chip is given in Eq. (9)'s section. The 2^{24} factor converts the floating-point composite coefficient into an integer for FPGA computation, and round() uses Banker's rounding to minimize quantization error.

With this design, the FPGA completes all quantization-related computation via one int64 multiplication followed by a 24-bit arithmetic right shift. It completely removes the demand for on-chip floating-point computing units [15], [25].

D. BIT-WIDTH SENSITIVITY ANALYSIS

To justify the choice of INT8 for a genuinely all-integer datapath, two bit-width sweeps are conducted at 16/8/6/4 bits on the FP32 model, as reported in Table 7 and Fig. 5. The full-integer sweep quantizes both weights and activations and is the primary evidence, since it reflects what the hardware actually executes; the weight-only sweep, in which activations remain in FP32, is included only as a contrast. Bit-width is a design choice, so both sweeps are evaluated on the validation set and never touch the test set; the activation ranges of the full-integer sweep are calibrated on the training set. The reference FP32 validation Macro-F1 is 0.8510. Consequently, the 8-bit full-integer PTQ point reported here (0.7843) is a validation-set figure and differs from the 0.7340 test-set PTQ value in Table 5: the two are consistent in showing that post-training quantization alone is inadequate, and differ only in evaluation set because bit-width selection is a validation-set decision whereas Table 5 reports the final test-set deployment comparison.

Three conclusions follow from Table 7.

First, the weight-only sweep is misleading on its own: it degrades gracefully and still reaches 0.7461 even at 4 bits,

TABLE 7. Bit-width sensitivity of the proposed 1D-CNN (144-dim, fold CF_9, validation set).

Bit Width	Full-Int. (PTQ)	Full-Int. (QAT)	Weight-Only
16-bit	0.8511	—	0.8511
8-bit	0.7843	0.8514	0.8506
6-bit	0.5333	0.8234	0.8460
4-bit	0.3997	0.5603	0.7461

which would wrongly suggest that very low bit widths are viable. The full-integer sweep tells the opposite story, showing that a weight-only study cannot on its own establish a full-integer Pareto point. The gap between the two curves shows that activation quantization, not weight quantization, is the dominant bottleneck for this heterogeneous LOB datapath.

Second, the accuracy curve for full-integer quantization should be interpreted alongside quantization-aware fine-tuning results. Under post-training quantization the accuracy collapses below 8 bits (0.7843 at 8-bit down to 0.5333 at 6-bit and to chance at 4-bit), but this cliff is an artifact of PTQ rather than an intrinsic bound: QAT recovers the 8-bit point to 0.8514 (matching the FP32 validation reference of 0.8510) and the 6-bit point to 0.8234, so under QAT the cliff shifts down to between 6 and 4 bits (4)-bit QAT reaches only 0.5603). In other words, 8-bit QAT is essentially loss-free, 6-bit QAT already incurs a visible loss of about 2.8 points but remains far above the PTQ cliff, and 4-bit collapses even under QAT. Accuracy alone therefore does not sharply force 8 over 6 bits; the deployed width is instead fixed by hardware: on the target DSP48E2, an 8-bit datapath is byte-aligned and packs two INT8 multiply-accumulates per DSP slice, whereas a sub-byte 6-bit width is not natively packed, does not reduce byte-addressed BRAM, and would add unpacking logic, so 6 bits give no proportional hardware saving while sacrificing accuracy. INT8 is therefore the hardware-aligned operating point: the lowest byte-aligned full-integer width that both preserves accuracy and maps efficiently onto the device, with sub-byte and ternary-weight datapaths, which have shown viability for low-precision signal classification [35], left to a future sub-byte-optimized design. All sweep values in Table 7 are validation-set Macro-F1; the deployed QAT model attains a test Macro-F1 of 0.8331, obtained with the per-channel-input weight-folded fixed-point export rather than the per-tensor fake-quant simulation of the sweep, so the two differ slightly in absolute terms while agreeing on the cross-bit-width trend.

Third, INT8 sits at the intersection of the accuracy cliff analyzed above and the DSP-efficiency cliff quantified by the INT8-versus-INT16 hardware ablation in Section VI-F (where widening to INT16 forfeits the DSP48E2's two-INT8-per-slice packing), making it the Pareto-optimal bit width for this edge-FPGA LOB accelerator.

V. FPGA ACCELERATOR DESIGN

This section elaborates the hardware implementation for the INT8 model from Section IV. The quantized weights, per-channel scales and zero points, and precomputed fixed-point requantization multipliers exported from the QAT flow are imported into the Vitis HLS project, then synthesized, placed and routed for the Xilinx Kintex UltraScale KU040 (xcku040-ffva1156-2-e). All variants share a single fixed-point arithmetic core and differ only in how it is folded and replicated. Two families of operating points are studied:

- **Latency-Optimized Architecture (LO-Arch):** a fully-unrolled single-stream design that minimizes single-sample latency for latency-critical, single-instrument inference.
- **Throughput-Optimized Architecture (TO-Arch):** a folded design whose engine can be spatially replicated, trading single-sample latency for higher aggregate throughput in multi-instrument scenarios.

Both architectures run entirely on-chip without FPUs, and their classification outputs match the PyTorch INT8 reference, with per-logit deviations bounded to a few INT8 quantization levels.

The rest of this section is structured as follows. Section V-A states three hardware design constraints derived from the research objectives in Section I. Section V-B specifies the numerical conventions of the datapath. Section V-C describes the fixed-point arithmetic core shared by all variants. Section V-D details the implementation of the residual 1D-CNN via six cascaded HLS layer functions. Section V-E and Section V-F present the LO-Arch and TO-Arch designs. Section V-G conducts a controlled DSE that decouples folding strategy from stream count and reports, within the measured configurations, the latency–resource trade-off across the folding family. Finally, the end-to-end experimental results are presented in Section VI.

A. DESIGN GOALS AND HARDWARE CONSTRAINTS

The accelerator design is constrained by three deployment requirements outlined in Section I, which are strictly followed throughout the software pipeline development in Section IV.

- **On-Chip-Only Weight Storage:** All 103.4 KB of INT8 parameters reside in on-chip BRAM with no off-chip DDR access during inference, removing a major source of latency jitter. This footprint is negligible against the 2.64 MB BRAM capacity, and after placement and routing the network occupies only a few 36-Kb RAMB36 BRAM tiles; on-chip utilization is therefore bounded by logic and DSP resources under unrolling rather than by weight storage.
- **Single-Sample Streaming Inference:** Unlike batch-based GPU inference, each edge LOB snapshot is processed individually at a fixed batch size $B = 1$ over a memory-mapped AXI interface: a $(10, 144)$ INT8 input tensor flows through a fully feed-forward datapath without control dependencies and yields three INT8 logits.
- **Full-Integer Arithmetic:** No floating-point IP cores are instantiated; all multiplication, accumulation and requantization use INT8/INT32 fixed-point arithmetic. Every requantization module follows the round-half-to-even (Banker's) rule of Section III-H, so the hardware reproduces the classification decisions of the PyTorch quantization reference.

The fully-unrolled LO-Arch delivers the minimal single-stream latency but is the most resource-intensive operating point. For higher aggregate throughput, the arithmetic core is folded to reduce per-engine resources and then spatially replicated, letting multiple independent, jitter-isolated streams share one chip at lower per-engine power for multi-instrument monitoring. The full latency–resource trade-off is characterized in Section V-G.

B. NUMERICAL CONVENTIONS

To remove ambiguity in the fixed-point datapath, in particular reconciling the signed `int8_t` storage with the unsigned $[0, 255]$ activation range, the accelerator follows a fixed set of numerical conventions. Activations are physically stored in `int8_t` bytes but interpreted as unsigned integers in $[0, 255]$, the real value used in arithmetic being the unsigned byte minus a zero point Z. The 144 network inputs use per-channel asymmetric quantization, so each input channel carries its own zero point $Z_{\text{in}}[c]$; the corresponding per-channel input scales have been folded into the first-layer weights during export, as described in Section IV-C, so no explicit per-channel input scaling remains on chip. Weights, in contrast, use per-channel symmetric quantization with $Z_w = 0$ and range $[-127, 127]$. Each layer accumulates in `int32` as $acc = \sum_c (x_{\text{int}} - Z_{\text{in}})\, w_{\text{int}}$, after which requantization multiplies by a per-output-channel integer $M[c]$ and arithmetically right-shifts by FRAC_BITS $= 24$, then adds the output zero point and clips: ReLU-fused layers clip to $[0, 255]$, whereas the residual convolution `res_conv_b`, which has no fused ReLU, uses the signed range $[-128, 127]$ so that it can emit negative values into the residual sum. Finally, the residual addition rescales its two inputs to a common output scale before summation, and the global average pool uses a fixed-point division by ten; both are detailed in Section V-C. All rounding is round-half-to-even.

C. FIXED-POINT ARITHMETIC KERNEL

All network arithmetic is performed by a unified shared core, instantiated once and inlined into each layer. It integrates three operators, implemented as C++ inline functions under the HLS INLINE directive.

1) PER-LAYER REQUANTIZATION (REQ_INT)

Convolutional and fully-connected layers use INT8 MAC as the core operation, and the INT32 accumulation is

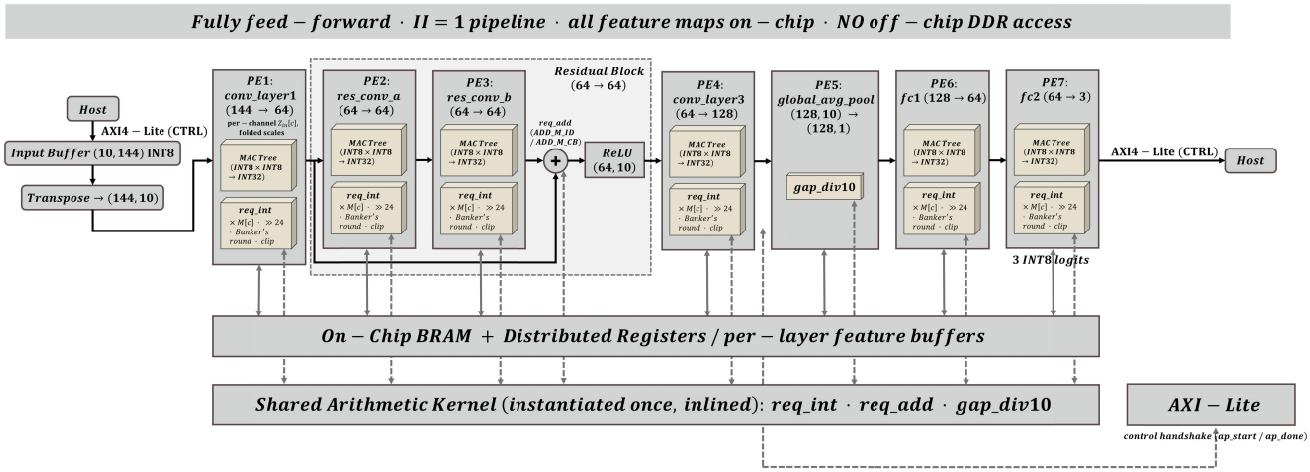


FIGURE 6. On-chip HLS datapath of the residual 1D-CNN accelerator showing the fully feed-forward pipeline stages.

requantized back to INT8. Let $acc = \sum(x_{\text{int}} - Z_{\text{in}})\, w_{\text{int}}[c]$ be the INT32 accumulation for output channel c; the input zero point Z_{in} is subtracted before accumulation, and symmetric weight quantization gives $Z_w = 0$. The requantization is formulated as:

$$y = \text{clip}\left(\text{round}_{\text{HE}}\left(\frac{\text{acc}\cdot M[c]}{2^{24}}\right) + Z_{\text{out}}, q_{\min}, q_{\max}\right) \quad (9)$$

The fixed-point multiplier is defined by:

$$M[c] = \text{round}\left(\frac{S_{\text{in}} \cdot S_{\text{w}}[c]}{S_{\text{out}}} \cdot 2^{24}\right) \quad (10)$$

The fixed-point multiplier $M[c]$ is pre-calculated offline by the weight export script introduced in Section IV-C5. Here, round_{HE} denotes round-half-to-even (Banker's rounding). The clipping range $[q_{\min}, q_{\max}]$ is configured as $[0, 255]$ for ReLU-fused layers and $[-128, 127]$ for all other layers.

In hardware this requires only a 64-bit multiplication, an arithmetic right shift and a conditional rounding correction—no dividers or FPUs. The 24-bit fractional precision (FRAC_BITS = 24) is the smallest power-of-two shift width that keeps classification consistent with the PyTorch FBGEMM INT8 backend across the full test set, with only negligible per-logit offsets.

Banker's rounding is validated by the ablation in Section VI-F: replacing it with plain truncation saves LUT and FF resources but breaks consistency with the PyTorch INT8 reference, amplifying error along the residual branch and degrading Macro-F1.

2) RESIDUAL-BRANCH SCALE ALIGNMENT (REQ_ADD)

As specified in Section III-E, the two input paths of the residual addition node hold independently optimized INT8 scales S_a and S_b, so direct integer addition across mismatched scales would overflow. The accelerator therefore unifies both paths to the output scale S_{out} before summation:

$$y = \text{clip}\left(\text{round}_{\text{HE}}\left(\frac{a\cdot M_a + b\cdot M_b}{2^{24}}\right) + Z_{\text{out}}, 0, 255\right) \quad (11)$$

Algorithm 1 Banker's Rounding for GAP Division

```
q = sum/10
r = sum%10
if r > 5:
    return q + 1
else if r < 5: 6:    return q
else:
    if q mod 2 = 0:
        return q
    else:
        return q + 1
```

In this equation, M_a and M_b are fixed-point multipliers that convert S_a and S_b to the unified output scale S_{out}. These precomputed multipliers are stored in the weight header file, denoted as ADD_M_ID for the identity shortcut branch and ADD_M_CB for the convolutional residual branch.

3) POOLING DIVISION (GAP_DIV10)

The GAP layer reduces the temporal dimension over a window of $T = 10$. Since division by the non-power-of-two 10 is inefficient in fixed-point hardware, the implementation computes quotient and remainder explicitly and applies Banker's rounding when $r = 5$, eliminating on-chip dividers, as presented in Algorithm 1.

D. NETWORK-LAYER MAPPING

As illustrated in Fig. 6, the inference pipeline is decomposed into six stages, each mapped to a dedicated HLS function: `conv_layer1`, `res_conv_a`, `res_conv_b` and the `res_block` adder, `conv_layer3`, `global_avg_pool` and the two fully-connected layers `fc1_layer` and `fc2_layer`, with `res_conv_b` left un-fused so it can emit signed values into the residual sum. Fixed-width convolution ensures all inter-layer buffers have compile-time-constant

sizes, so Vitis HLS maps the entire dataflow to registers and on-chip BRAM without dynamic memory management.

The top-level function `lob_inference` receives the (10, 144) input tensor and returns the three logits through a single AXI4-Lite control interface (bundle `CTRL`) over one subordinate port. Internally, the input is transposed to the (144, 10) channel-first layout required by the first convolutional layer. All weights and intermediate buffers reside on chip, so no off-chip memory access occurs during a single inference pass.

E. LO-ARCH: LATENCY-OPTIMIZED ARCHITECTURE

LO-Arch targets the most stringent operating point in the design space, with the explicit goal of minimizing single-stream latency. This architecture is tailored for use cases where a single edge FPGA monitors a single trading symbol, as well as canonical HFT scenarios where every microsecond of latency directly determines competitive trading advantages.

1) FULL SPATIAL UNROLLING

LO-Arch employs the most aggressive set of HLS optimizations feasible under available on-chip resource budgets:

- The directive `#pragma HLS ARRAY_PARTITION variable=...complete dim=0` is applied to all layer input, padding and intermediate buffers, splitting each array into independent ports for parallel access.
- `#pragma HLS UNROLL` is inserted in preprocessing loops for input channel rearrangement to enable hardware reuse of padding and zero-offset arithmetic logic.
- The output-channel loops of convolutional and fully-connected layers carry a loop-level `#pragma HLS PIPELINE II=1`, with an initiation interval of 1 enforced at the loop level. The top-level inference function is deliberately not pipelined: the design is single-shot and latency-optimized, so successive inferences do not overlap and the effective throughput equals 1/latency rather than one result per clock cycle.

These optimizations fully parallelize the arithmetic within each layer while layers execute sequentially as a pure feed-forward dataflow. With the 144-channel input, all layers including the first convolution are fully unrolled, so the single-sample latency is dominated by the unrolled body rather than by any folded loop. Under a 100 MHz clock, the accumulated pipeline depth yields an end-to-end latency of 57 μs, with all data confined to on-chip registers and BRAM and no off-chip access at any step.

Because the datapath is purely feed-forward with no data-dependent branches, its cycle count is fixed and independent of the input, so the latency is deterministic by construction rather than merely observed to be stable. Implemented on the Kintex KU040 at 100 MHz, the placed-and-routed LO-Arch meets timing with a worst negative slack of +0.520 ns and occupies 37.6% of LUTs, 56.0% of DSPs and under 1% of BRAM blocks, as listed in Table 8; full accuracy, power and cross-platform results appear in Section VI. The Power column of Table 8 lists Vivado post-route vectorless chip-level estimates, used only for relative comparison within the folding family; they are on a different basis from the whole-board DT550 measurement (3.11 W) reported for the deployed LO-Arch in Section VI-D and are not directly comparable in absolute terms. The 100 MHz target is a conservative choice that guarantees timing closure across all folding variants and matches the JTAG-AXI verification environment; the positive slack across the family (Table 8) indicates headroom for a higher clock, and raising the frequency to further reduce latency is left as future work, as 57 μs already meets the target HFT budget.

F. TO-ARCH: THROUGHPUT-OPTIMIZED ARCHITECTURE

TO-Arch targets multi-stream concurrent workloads, letting a single FPGA process multiple independent LOB samples in parallel. When aggregate throughput rather than single-sample latency is the objective, the folded engine uses far fewer resources than LO-Arch and can be replicated across streams within the device budget (Section V-G). It employs a time-sharing folding strategy that reuses arithmetic units across time slots, with folding factors matched to each layer's channel width so that a single global factor does not force wide and narrow layers to the same parallelism [32].

1) DIMENSION-AWARE MIXED-FACTOR FOLDING

A single global folding factor forces every layer to the same degree of parallelism, which is inefficient when the layers differ in width. The network adopts a channel dimension progression of 144 $\rightarrow$ 64 $\rightarrow$ 128. A factor small enough to divide the 144-channel first layer, such as 8, leaves the deeper 64- and 128-channel layers under-parallelized: with uniform factor 8 the whole engine runs at 1324 μs. Assigning each layer a factor matched to its channel width instead raises the parallelism of the deeper layers without over-committing resources on the widest one.

To address the above limitations, folding coefficients are optimized to align with the inherent channel count of each layer:

- `conv_layer1` (144 input channels): a folding factor that divides 144, such as 8, is used, so feature segmentation introduces no control overhead.
- `res_conv_a`, `res_conv_b`, `res_block` and `conv_layer3` (64/128 channels): A unified folding and unfolding factor of 16 is applied. Both 64 and 128 are divisible by 16, forming a regular cyclic memory layout.

Two implementation techniques ensure reliable deployment of this dimension-aware mixed folding:

- The `#pragma HLS PIPELINE` constraint is applied to the innermost loop of input channels, which differs from the output-channel pipelining adopted in LO-Arch. This reduces the complexity of memory

TABLE 8. Controlled folding design-space exploration on KU040 (100 MHz, 144-dim INT8).

Variant	Streams	Folding	Latency	WNS (ns)	LUT	DSP	Power (W)	Feasible	Source
LO-Arch	1	Fully unrolled	57 μs	+0.520	91,233 (37.6%)	1,076 (56.0%)	2.288	✓	post-route
Sweet-spot	1	conv1 factor 24	250 μs	+0.368	64,598 (26.7%)	808 (42.1%)	1.926	✓	post-route
Uniform-8	1	Uniform factor 8	1324 μs	+2.940	12,518 (5.2%)	83 (4.3%)	1.005	✓	post-route
Uniform-8 (2 streams)	2	Uniform factor 8	1324 μs	+2.907	22,175 (9.2%)	166 (8.7%)	1.163	✓	post-route
Mixed-8/16	1	Mixed factor 8 / 16	1038 μs	+1.745	19,491 (8.0%)	147 (7.7%)	1.129	✓	post-route
Mixed-8/16 (4 streams)	4	Mixed factor 8 / 16	1038 μs	—	350,268 (144%)	392 (20%)	—	×	HLS est.

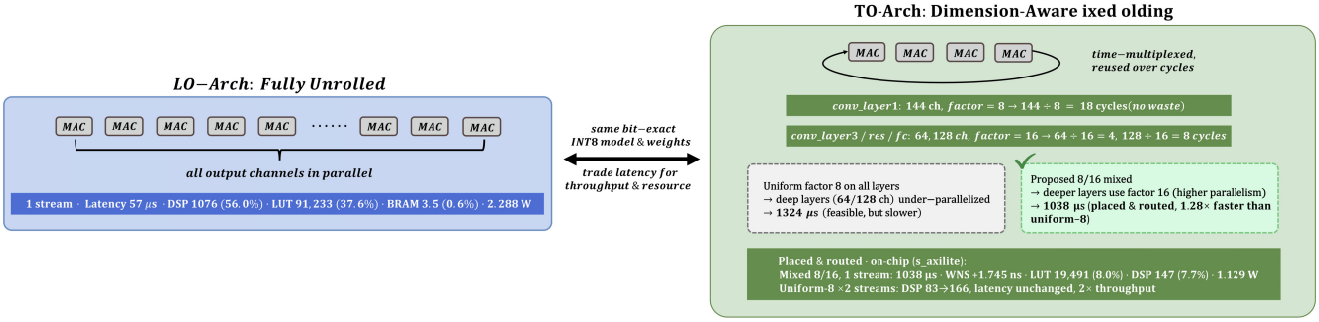


FIGURE 7. Architectural comparison of the fully-unrolled LO-Arch and the dimension-aware mixed-factor folding TO-Arch.

access scheduling and avoids large-scale cross-channel rotation multiplexers.

- Custom loop partitioning is adopted instead of full partitioning or block partitioning. Feature channels indexed by i and $i+$factor are retrieved from independent BRAM ports simultaneously. This mechanism eliminates port contention and mitigates excessive register overhead caused by single-channel data buffering.

By raising the folding factor of the deeper 64/128-channel layers from 8 to 16, the mixed-factor engine runs at 1038 μs versus 1324 μs for uniform factor 8, a 1.28× speed-up obtained by matching folding factors to channel widths rather than by adding resources. This scheme is placed and routed on the KU040, not merely estimated in HLS: it closes timing at +1.745 ns and occupies 8.0% of LUTs (19,491), 7.7% of DSPs (147) and 43.5 BRAM tiles at a vectorless 1.129 W (Table 8)—a footprint comparable to uniform factor 8, confirming on hardware that the speed-up comes from the folding-factor assignment itself, not a larger resource envelope. Fig. 7 contrasts the fully-unrolled LO-Arch with the mixed-factor folding TO-Arch.

2) SPATIAL REPLICATION OF FOLDED ENGINES

Folding optimization effectively reduces the resource overhead of a single inference engine. On this basis, the top-level function uses the `#pragma HLS UNROLL` directive for spatial unrolling, instantiating two independent folded inference engines to support dual-stream parallel processing.

The two parallel inference engines adopt fully separated partitioned storage for input data, network weights and output results, completely eliminating BRAM port contention. The top level simply wraps the single-engine `lob_inference_single` in a `NUM_STREAMS` loop carrying `#pragma HLS UNROLL`, with the input and output arrays partitioned along the stream dimension, so the AXI interface is expanded to receive N independent (10, 144) input tensors and output N groups of three INT8 logits in parallel.

Because each replicated engine has private partitioned storage, the streams are jitter-isolated and the single-sample latency is unchanged by replication; only the aggregate throughput scales with the number of streams. The concrete latency and resource figures for the folded single- and two-stream configurations are reported in Table 8.

G. DESIGN SPACE EXPLORATION

This section explores the design space by varying the two design variables—folding strategy and stream count—along separate axes rather than simultaneously, so each effect is attributed cleanly. The folding axis (fixed at one stream) sweeps full unrolling, conv1 factor 24 and uniform factor 8; the replication axis (fixed at uniform factor 8) sweeps one

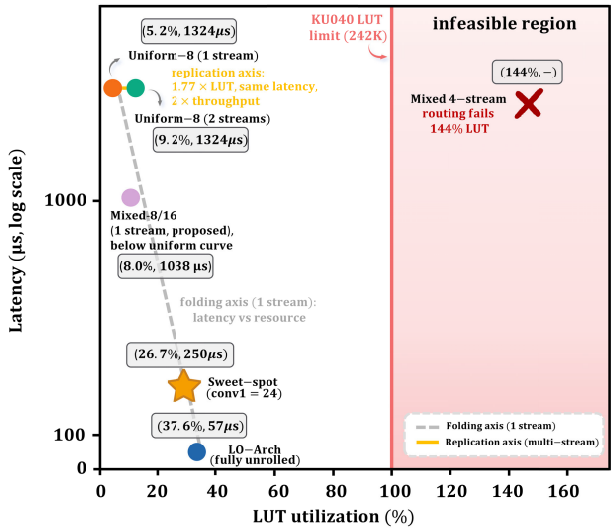


FIGURE 8. Design space exploration: latency versus LUT utilization for folding architectures on KU040.

and two streams. All feasible points are synthesized, placed and routed at 100 MHz on the KU040; results are given in Table 8 and Fig. 8. Latency is set by the folding strategy and is independent of stream count.

Analysis of the post-route data yields three conclusions, each stated only within the measured configurations.

- **Folding axis: latency trades off monotonically against resources.** At one stream, increasing the folding depth from full unrolling to conv1 factor 24 to uniform factor 8 raises latency from 57 to 250 to 1324 μs while lowering LUT usage from 37.6% to 26.7% to 5.2% and DSP usage from 56.0% to 42.1% to 4.3%. Fully-unrolled LO-Arch is thus the latency extreme and uniform folding the resource-efficient extreme, with the conv1-factor-24 point as an intermediate sweet spot; all three meet timing with positive slack. The dimension-aware mixed-8/16 point does not lie on this uniform curve: at a footprint close to uniform factor 8 (8.0% LUTs, 7.7% DSPs, 1.129 W) it delivers 1038 rather than 1324 μs, moving the operating point below the curve by matching folding factors to channel widths instead of spending extra logic—the specific advantage of the proposed scheme over a generic depth–resource trade.
- **Replication axis: throughput scales linearly at constant latency (measured 1→2 streams).** Holding uniform factor 8 fixed and replicating from one to two streams leaves the single-sample latency unchanged at 1324 μs and doubles DSP usage from 83 to 166 (exactly 2×) and LUT usage from 5.2% to 9.2%, so aggregate throughput rises from 755 to 1510 inferences per second while both configurations meet timing. This single measured 1 → 2 replication shows the expected doubling with no latency penalty and jitter isolation. The replication axis is demonstrated on the uniform-factor-8 engine because its small single-engine footprint (5.2% LUTs) is what leaves headroom for replication; the mixed-factor folding and the spatial replication are orthogonal mechanisms and can in principle be composed, but a post-route mixed-folding multi-stream point is not separately reported here, since the deeper mixed engine leaves less replication headroom on this device (the four-stream mixed estimate already exceeds capacity, below). A dedicated post-route characterization of composed mixed-folding-plus-replication configurations on a larger device is left as future work.
- **Device-capacity limit.** Not all combinations fit: replicating the mixed-folding engine to four streams already requires an estimated 144% of the KU040 LUT capacity at the HLS synthesis stage (350,268 LUTs against the device's 242,400), so it exceeds the device before place-and-route is even attempted and is therefore infeasible on this part. This four-stream figure is a pre-implementation HLS synthesis estimate—reported as such in Table 8 and not directly comparable to the post-route values of the feasible variants—but a LUT requirement already above 100% of capacity is sufficient on its own to rule the configuration out. Notably, the resource growth from one to four mixed-folding streams is non-linear rather than a clean 4×: LUTs scale super-linearly (interconnect, multiplexing and per-engine control replicate faster than the datapath) while DSPs grow sub-linearly (synthesis shares and packs multiplier resources), which is precisely why aggregate throughput cannot be extended indefinitely by naive spatial replication and motivates the folding-aware design. No single configuration dominates; the appropriate operating point depends on whether single-sample latency or aggregate throughput is prioritized, and the Pareto and scaling statements above hold only within the measured configurations.

Two additional hardware ablations, comparing INT8 with INT16 and Banker's rounding with truncation, are reported in Section VI-F.

VI. EXPERIMENTAL RESULTS AND EVALUATION

This section evaluates the accelerator across functional and on-board validation, FPGA resource and latency, power and energy efficiency, cross-platform comparison against a GPU, and two hardware ablations. All FPGA results are post-place-and-route on the Xilinx Kintex UltraScale KU040 at 100 MHz; all software experiments use the strict chronological FI-2010 partition of Section IV.

A. POSITIONING AGAINST PRIOR LOB PREDICTORS

To position the accelerator within FI-2010 LOB prediction, Table 9 separates two kinds of numbers. Zone (a) contains models re-run in this work under one identical protocol—the leakage-controlled anchored fold CF_9 with the same 144-dimensional input, labels, normalization and purge/embargo of Section IV-A4—so these Macro-F1 values are

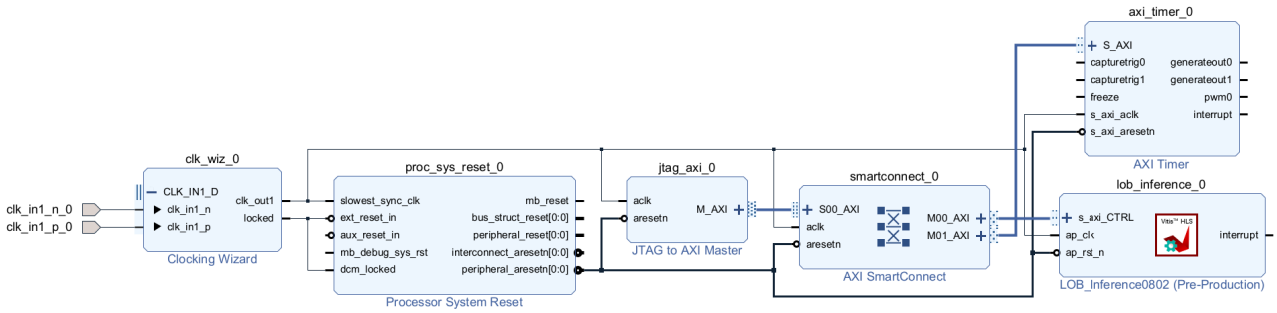


FIGURE 9. Vivado block design of the on-board verification system: a JTAG-to-AXI manager drives the `lob_inference` IP over an AXI smartconnect on the KU040.

TABLE 9. Positioning against prior LOB predictors (FI-2010, k=100, NoAuction, Z-score).

Model	Macro-F1	Edge-FPGA Deployable
(a) Same protocol (anchored fold CF_9, re-run in this work)		
DeepLOB [1]	0.8789	No
Proposed (FP32)	0.8509	Yes
TLOB [44]	0.8417	No
Proposed (INT8)	0.8331	Yes
MLP	0.8243	Yes
LSTM	0.7986	No
(b) Literature values (different protocol; not comparable)		
B(TABL) [43]	0.6886	No
C(TABL) [43]	0.7352	No
DeepLOB [1]	0.7658	No

directly comparable. Zone (b) quotes literature numbers obtained under a different protocol—40-dimensional raw inputs, averaged over all nine folds, and without the purge/embargo leakage control—so they are background only and not comparable to zone (a). For a like-for-like comparison, the zone-(a) baselines keep their published network architectures but are re-run under the common labeling, normalization and purge/embargo protocol of this work (Sections III-B, III-C and IV-A4) rather than each original paper's setup. In particular, since TLOB's original work adopts its own labeling scheme, its architecture is reused here but evaluated under the shared FI-2010 label protocol, so the reported TLOB Macro-F1 reflects the common protocol.

Within the comparable zone, DeepLOB attains the highest Macro-F1 (0.8789), slightly above the proposed FP32 model (0.8509), while TLOB (0.8417), MLP (0.8243) and LSTM (0.7986) fall below it; the proposed INT8 model reaches 0.8331 after quantization. The modest gap to DeepLOB stems from its recurrent LSTM stages, hard to map to a deterministic fully-unrolled pipeline, while TLOB relies on attention and Softmax costly on a resource-constrained edge FPGA. The contribution is therefore system-level rather than an accuracy claim: among these models the proposed design is the only one running entirely on a standalone edge FPGA at microsecond deterministic latency.

B. FUNCTIONAL CONSISTENCY AND ON-BOARD VERIFICATION

Functional verification is two-level. First, HLS C/RTL co-simulation confirms argmax classifications identical to the PyTorch INT8 reference with only minor logit offsets, validating the 32-bit accumulation and Banker's rounding datapath.

Secondly, the placed-and-routed design is validated on a physical KU040 board. As shown in Fig. 9, the verification system is a Vivado block design in which a JTAG-to-AXI manager drives the `lob_inference` IP over an AXI SmartConnect. The entire held-out test set of 31,828 samples is streamed into the on-chip core and each output compared against the PyTorch INT8 reference: the two agree on all 31,828 samples, i.e., the board is bit-exact with the reference over the full test set, so any accuracy gap relative to software is due to quantization, not to the hardware.

On this full 31,828-sample on-board set the raw classification accuracy is 83.12%, with a 95% Wilson confidence interval of 82.71% to 83.53% and a Clopper–Pearson interval of 82.71% to 83.53%. This matches the full-test-set INT8 top-1 accuracy of 83.12% (corresponding to the deployed Macro-F1 of 0.8331 in Table 5), confirming exact software–hardware agreement over the whole deployment-fold test day. Throughout this paper the deployed model's headline score 0.8509 is a Macro-F1 and its 0.8496 counterpart is the corresponding top-1 accuracy of the same FP32 model, the two being different metrics on the same run rather than different results; INT8 figures follow the same convention (Macro-F1 0.8331, accuracy 0.8312). The board run uses one representative synthesized variant—the fully-unrolled 57 μs LO-Arch bitstream, i.e. the same design whose whole-board power is measured at 3.11 W in Section VI-D, so the 31,828-sample bit-exact result, the 57 μs latency and the 3.11 W power all refer to one and the same bitstream—and the result transfers to the whole family because all variants share the same `weights.h` and

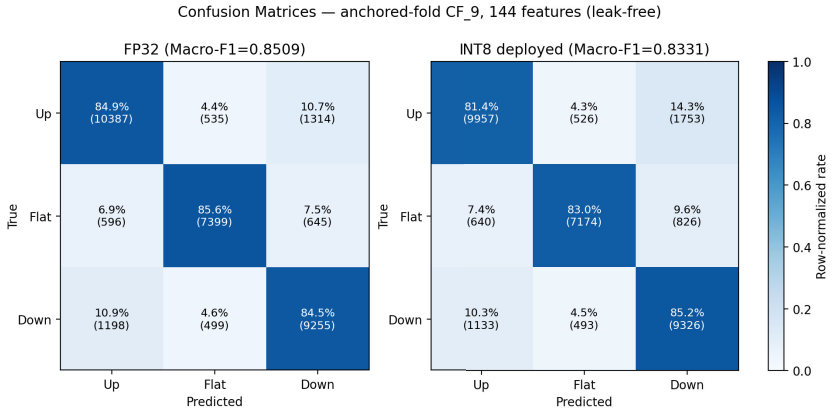


FIGURE 10. Confusion matrix of the deployed INT8 model on the FI-2010 test set.

fixed-point core: folding and replication change only the loop schedule and engine count, not the sequence of integer multiply–accumulate, requantization and rounding applied to a sample. Bit-exactness is therefore a property of the shared datapath, not of any folding factor, so the LO-Arch and the folded configurations reproduce the same predicted classes as the PyTorch INT8 reference, as confirmed in C/RTL co-simulation for each variant. The JTAG-to-AXI harness used here validates full functional and numerical equivalence over the entire test set, but it exercises the core one sample at a time under host control rather than as a sustained line-rate stream; a DMA- or replay-driven streaming harness that measures back-to-back throughput under a realistic feed is left as future work. Because the core's cycle count is input-independent and its output depends only on the current window and not on arrival timing, such a harness would not change the predicted classes or the per-sample core latency, only the surrounding data-movement schedule. Two points make this deferral a matter of engineering scope rather than a limitation of the core. First, the streaming front end requires no change to the core itself: the target board supports AXI-DMA and the inference core already exposes a standard streaming AXI interface, so a DMA-/replay-driven data pump can be attached directly. Second, a faithful line-rate test requires a timestamped raw market-data feed with realistic tick arrival times, whereas FI-2010 is an offline snapshot dataset without arrival timestamps, so any imposed replay rate would be synthetic; a meaningful streaming/tick-to-trade evaluation therefore belongs with the future integration of a live feed handler and MAC-layer front end.

The core latency is deterministic by construction: the branch-free fixed-point datapath has an input-independent cycle count, and timing closure (Section V-G) guarantees it is met at 100 MHz. The authoritative latency is therefore the cycle-accurate co-simulation schedule times the closed clock period: as the cycle count cannot vary with input, this is an exact value rather than a sampled mean, and a distribution of repeated measurements would add no information for a datapath whose every execution takes identical cycles. The on-board JTAG-AXI runs corroborate this at the order-of-magnitude level but are not the primary source, since the transfer and host handshake around the core would dominate a wall-clock stopwatch.

The per-class confusion matrix of the deployed INT8 model on the full test set is shown in Fig. 10.

C. FPGA RESOURCE UTILIZATION AND LATENCY

As summarized in Table 8, after placement and routing the fully-unrolled LO-Arch delivers a deterministic single-stream latency of 57 μs—tens of microseconds, and more than an order of magnitude below the sub-millisecond regime of batch-one GPU inference—while occupying 56.0% of DSPs, 37.6% of LUTs and under 1% of BRAM blocks. Folding trades latency for resources along the family of Table 8: the conv1-factor-24 sweet spot runs at 250 μs (42.1% DSPs, 26.7% LUTs) and the uniform-factor-8 configuration at 1324 μs (4.3% DSPs, 5.2% LUTs), leaving ample room for stream replication. All routed configurations close timing with positive slack.

D. POWER AND ENERGY EFFICIENCY

Power for the deployed LO-Arch is measured directly on hardware: an external DT550 in-line DC power meter on the board's 12 V supply rail reads 3.11 W (fluctuating within 3.10–3.12 W) for the whole board during sustained on-board inference. This whole-board reading is dominated by the fixed platform overhead—four DDR4 devices, regulators and the cooling fan—so the inference core's own dynamic power (well under 1 W for an 86 K-parameter fully on-chip datapath that makes no off-chip DDR access) is small relative to it and below what an in-line rail meter can resolve against this baseline. 3.11 W is therefore treated as a conservative whole-board upper bound on the operating-point power rather than as the core's dynamic power, and the energy-efficiency ratios derived from it are correspondingly device-level upper-bound estimates. This whole-board figure

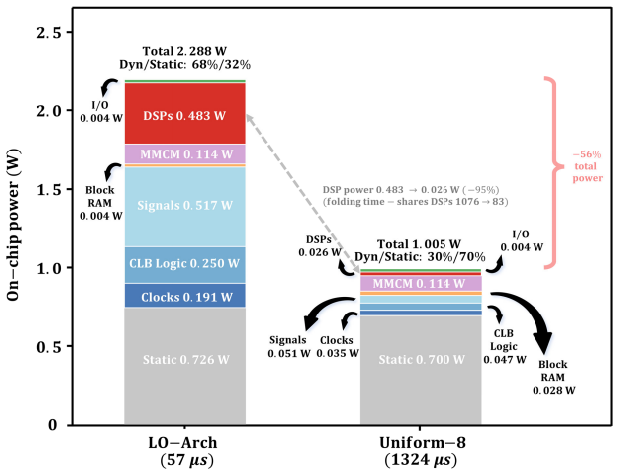


FIGURE 11. On-chip power breakdown of LO-arch and TO-arch (post-route chip-level estimates).

is on the same device-level basis as the GPU baseline (Section VI-E), whose power is likewise the total card draw, so the two are directly comparable. For the folding design-space study, power is reported as Vivado post-route vectorless chip-level estimates (medium confidence, ±10–20%): the conv1-factor-24 configuration at 1.926 W and the uniform-factor-8 configuration at 1.005 W, as broken down in Fig. 11. Chip-level estimates are used here deliberately: they isolate the fabric-power differences that the folding factors control, which would otherwise be masked by the fixed board-level overhead (DDR devices, regulators and the cooling fan) that dominates the whole-board reading. They are therefore not directly comparable in absolute terms to the whole-board measurement above and are used only for relative comparison within the folding family.

Energy efficiency, defined as inferences per second per watt (IPS/W) [28], [38], is used to evaluate computational efficiency. Table 10 compares the batch-one latency, power, throughput and energy efficiency of the deployed LO-Arch accelerator against the general-purpose GPU baseline at two FP32 software stacks—eager PyTorch and the optimized ONNX Runtime CUDA engine—both measured on an equivalent whole-device basis: the LO-Arch power is the DT550 in-line reading on the 12 V board rail and the GPU power is the physically measured steady-state value from `nvidia-smi`, rather than a nameplate rating. Reporting both GPU stacks bounds the comparison: LO-Arch is 23.8× more energy-efficient than the strongest optimized GPU engine and 41.6× than eager PyTorch. Both GPU rows use the same 12.39 W steady-state draw, as the two software stacks differ in latency but not measurably in steady-state board power (the difference is within the `nvidia-smi` measurement noise). The folding family's latency, resource and power trade-offs are reported separately in the design-space study (Table 8).

TABLE 10. Cross-platform latency, power and energy efficiency (batch = 1).

Platform	Latency (μs)	Power (W)	Throughput (IPS)	Energy Eff. (IPS/W)	Resource Eff. (IPS/kLUT)
RTX 5070 GPU (eager PyTorch)	594.96	12.39	1,681	135.7	—
RTX 5070 GPU (ONNX RT CUDA)	341	12.39	2,933	236.7	—
LO-Arch	57	3.11	17,544	5,641	192.3

E. CROSS-PLATFORM COMPARISON

GPU latency and power are measured over ten consecutive runs under steady-state operation using native SM_{120} (Blackwell) support; power is read from nvidia-smi.

At batch size 1, the NVIDIA RTX 5070 Laptop GPU achieves an inference latency of 594.96 μs at a physically measured steady-state power of 12.39 W (against a 9.93 W idle floor). Single-sample inference is the only working mode for tick-level high-frequency trading, and in this regime the GPU is dominated by kernel-launch and host–device synchronization overhead rather than by the arithmetic of the compact model; this is a workload mismatch rather than a raw-compute deficit. The GPU only amortizes these fixed overheads at much larger batch sizes: at batch size 4 the per-sample latency already falls to about 147 μs, but such batching aggregates independent ticks and is therefore irrelevant to latency-critical single-instrument trading—forming a batch of four ticks forces each sample to wait for up to three additional inter-tick intervals before inference even begins. This batch-1-versus-batch-4 scaling also quantifies the overhead directly: the whole-call wall-clock is essentially unchanged between batch 1 and batch 4 (median 563 μs versus 564 μs), which decomposes into a fixed per-call cost of about 563 μs and a per-sample arithmetic cost of only about 0.36 μs. At batch one, therefore, roughly 99.9% of the GPU latency is fixed kernel-launch and host–device synchronization/transfer overhead and only about 0.06% is arithmetic, so the 147 μs per-sample figure at batch 4 is an amortization artifact of dividing an essentially constant 564 μs call by four rather than a genuine latency reduction. The fully on-chip FPGA datapath removes this overhead entirely [28]. The GPU comparison is therefore presented as an illustrative workload-mismatch result rather than a definitive accelerator benchmark: a native INT8 GPU engine (e.g. TensorRT) could not be instantiated on the Blackwell-class device used here owing to a toolchain ABI incompatibility, so the claim that INT8 would not remove the fixed per-call overhead is supported by the measured FP16 result (which

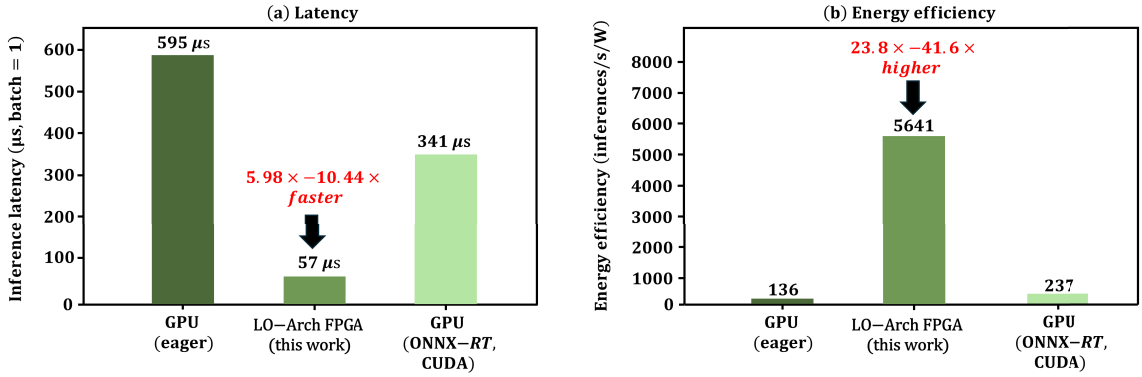


FIGURE 12. Cross-platform comparison at batch size one: (a) inference latency and (b) energy efficiency.

is *slower* than FP32, Table 11) rather than by a direct INT8-GPU measurement. The energy-efficiency advantage should be read in this light—as a device-level, batch-one deployment comparison at each platform's practical software stack, not an equal-precision arithmetic claim.

Because a general-purpose platform's latency is a distribution rather than a fixed value, Table 11 reports the full batch-one latency statistics over 5,000 iterations, together with optimized GPU-side and CPU-side inference engines requested to strengthen the software comparison. Two facts follow. First, the software baselines exhibit heavy right tails—the GPU p99 (995 μs) is 1.77× its median and the native-CPU p99 (560 μs) is 1.94× its median—whereas the FPGA core has an input-independent cycle count and therefore zero run-to-run variance ($\sigma = 0$), so its p99 equals its median at 57 μs. This tail behaviour, not merely the mean, is what disqualifies general-purpose platforms for deterministic microsecond inference. Notably, at batch one the CPU median (288 μs) is in fact lower than the GPU median (563 μs), because the compact model cannot amortize the GPU's fixed kernel-launch and host–device transfer overhead at a single sample; this does not favour either platform for the present comparison, since both exhibit heavy right tails (1.8–2.4× median-to-p99) and neither offers the input-independent, jitter-free latency the FPGA core provides. Second, optimizing the CPU path with ONNX Runtime lowers the median to 226 μs at a prediction-level agreement of 100% with the PyTorch reference (identical Macro-F1 of 0.8509), yet it is still 3.96× slower than the 57 μs core and retains a 2.4× median-to-p99 tail; a stronger software backend narrows but does not close the latency-and-determinism gap.

Optimizing the GPU path was likewise attempted along two axes. Graph optimization and operator fusion via the ONNX Runtime CUDA backend lower the GPU median from the 563 μs of eager PyTorch to 341 μs (agreement 99.997% with the reference), while half-precision (FP16) execution does not help and in fact raises the median to 619 μs. This apparent paradox is itself the key evidence: for an

TABLE 11. Batch-one latency distribution over 5,000 iterations (μs).

Platform	Median	p95	p99	Min	Max	Std
GPU (PyTorch, FP32)	563	751	995	489	1779	91
GPU (ONNX RT CUDA, FP32)	341	509	664	241	1266	87
GPU (PyTorch, FP16)	619	852	1292	444	2481	142
CPU (PyTorch, FP32)	288	466	560	214	961	64
CPU (ONNX RT, FP32)	226	327	535	96	1204	79
FPGA core (INT8)	57	57	57	57	57	0

86K-parameter model at batch one, the wall-clock is dominated by fixed per-call kernel-launch and host–device synchronization overhead rather than by arithmetic, so reducing arithmetic precision cannot shrink it—and the extra half-precision cast kernels even enlarge it slightly. The strongest optimized GPU engine (ONNX Runtime CUDA, 341 μs) therefore remains 5.98× slower than the 57 μs core with a heavy tail, and no arithmetic-level GPU optimization can close a gap that is structural rather than compute-bound. A dedicated TensorRT INT8 engine was also attempted but could not be initialized on the Blackwell-class device used here owing to a known ABI mismatch between the prebuilt ONNX Runtime TensorRT provider and the device's toolchain; given the precision-independence just demonstrated, it would not alter this conclusion. The 594.96 μs value used for the energy-efficiency ratios above is a ten-run steady-state average measured together with the GPU power; the independent 5,000-iteration distribution run of Table 11 gives a consistent mean of 588 μs (median 563 μs), confirming the two measurements agree to within about 1%.

To avoid overstating the advantage against a weakly optimized baseline, the comparison is reported as a range across GPU software stacks at the same FP32 precision: the

strongest optimized GPU engine (ONNX Runtime CUDA, median 341 μs) gives a 5.98× latency reduction and, at its 236.7 IPS/W, a 23.8× energy-efficiency advantage, whereas the eager-PyTorch baseline (594.96 μs) gives 10.44× and 41.6×. The two endpoints use slightly different statistics—341 μs is the median of the 5,000-iteration distribution run (Table 11), whereas 594.96 μs is the ten-run steady-state mean measured jointly with the GPU power draw and therefore used for the energy-efficiency ratio; the two statistics agree to within about 1% on the same platform (the eager run's own median is 563 μs, i.e. 9.88×), so the range is not an artifact of mixing statistics. Even against the fastest GPU engine available here, the core therefore remains almost an order of magnitude faster with far lower jitter. Precision is not the confound here—both GPU figures are FP32, and since the batch-one GPU is bound by kernel-launch and synchronization rather than arithmetic, quantizing it to INT8 would leave these fixed per-call costs, and hence its latency, essentially unchanged (the FP16 row of Table 11, which is *slower* than FP32, confirms this directly). The figures are whole-device deployment ratios, not equal-precision arithmetic-efficiency claims.

The fully-unrolled LO-Arch runs at a deterministic latency of 57 μs at a directly measured 3.11 W. Relative to the GPU baseline it thus achieves a 5.98–10.44× latency reduction and, on the equivalent whole-device power basis, a 23.8–41.6× improvement in energy efficiency (optimized ONNX Runtime CUDA to eager PyTorch, respectively) [38], as visualized in Fig. 12.

To test whether the small accuracy difference between the deployed INT8 model and its FP32 counterpart is systematic, a McNemar test on the paired predictions over all 31,828 on-board samples is performed. The FP32 and INT8 predictions disagree on 2,070 samples (1,327 versus 743 in the two off-diagonal cells), giving an exact McNemar $p < 0.001$ and a continuity-corrected $\chi^2 = 164.2$. The test therefore detects a systematic FP32-versus-INT8 difference, but its magnitude is small—about 1.83 percentage points of top-1 accuracy. This McNemar figure is expressed in accuracy because the test operates on per-sample correctness, and is the accuracy-domain counterpart of the 1.78-point Macro-F1 loss reported elsewhere (the two differ only because they are computed on different metrics); both lie within the range expected for full-integer quantization and do not affect the deployment conclusion. The very small p-value reflects the large paired sample size, which raises the test's statistical power, rather than a large accuracy gap; statistical detectability at this scale should not be read as practical importance.

F. HARDWARE ABLATION STUDIES

Two controlled ablations isolate two core hardware choices: the Banker's rounding datapath and the INT8 data width. Both use a common folded engine (conv1 factor 8, residual layers factor 16) and change one variable at a time, so the deltas are attributable. The three variants are C/RTL bit-exact against their software references. The resources in Table 12 are HLS post-synthesis (pre-implementation) estimates, reported here only for the *relative* comparison between the three variants under an identical flow; they are on a different basis from the post-route utilization of Table 8 (HLS synthesis systematically over-estimates LUTs, before place-and-route logic optimization), so the absolute LUT counts here are not directly comparable to the routed figures of the folding family and only the within-table deltas are meaningful.

TABLE 12. Hardware ablation on rounding logic and datapath precision (KU040, 100 MHz, matched folding; HLS post-synthesis estimates for relative comparison).

Variant	Datapath	Rounding	DSP	LUT
Baseline	INT8	Banker's	98	89, 125
Truncation ablation	INT8	Right-shift	98	78, 645
Extended precision	INT16	Banker's	153	95, 405

1) ROUNDING LOGIC: BANKER'S ROUNDING VERSUS TRUNCATION

Replacing the round-half-to-even (Banker's) requantization of Section V-C1 with a plain arithmetic right shift, while holding the folding structure fixed, lowers LUT usage from 89, 125 to 78, 645 (a saving of about 11.8%) and flip-flop usage by roughly a third, with DSP usage unchanged at 98, since the multipliers are identical and only the remainder-handling logic is removed.

However, truncation breaks bit-exactness with the PyTorch INT8 reference: its systematic downward bias accumulates along the residual branch and the long multiply–add chains. On the full CF_9 test set it changes 6.82% of predictions and lowers the Macro-F1 from 0.8331 to 0.8255, a drop of 0.76 points. As the truncated variant is no longer numerically identical to the reference, it cannot be validated bit-exactly against PyTorch, and the modest LUT/FF saving does not justify losing that guarantee; Banker's rounding is therefore retained.

2) DATAPATH PRECISION: INT8 VERSUS INT16

Widening the datapath from INT8 to INT16 (64-bit accumulators, all else equal) raises DSP usage by 56% (98 to 153) and FF usage by about 80%, while LUTs rise from 89, 125 to 95, 405. This follows from the DSP48E2 architecture: a single slice packs two INT8 multiply–accumulate operations, whereas a 16-bit multiply occupies a whole slice and wide accumulators require extra cascading. Crucially, this INT16 variant uses the same 8-bit-trained weights and is bit-exact with the INT8 reference, so it yields no accuracy gain. Combined with the sub-8-bit accuracy degradation analyzed in Section IV-D, this confirms INT8 as the Pareto-optimal width: widening beyond it only spends DSP and FF resources for zero accuracy benefit.

VII. DISCUSSION

A. DEPLOYMENT SCENARIOS

The fully-unrolled LO-Arch (57 μs, a measured 3.11 W) is suited for single-instrument, latency-critical nodes where minimizing per-sample latency is decisive. The folded configurations (for example the 250 μs sweet spot and the 1324 μs uniform-factor-8 point) suit multi-instrument monitoring, where a lower per-engine resource and power budget and strict per-stream latency isolation matter more than the latency of any single stream. To indicate what the accuracy means in use, the deployed INT8 model attains a recall of about 0.81 (upward) and 0.85 (downward) on the full test set (Fig. 10; the per-class recalls in this discussion are from the deployed fixed-point model, whereas the slightly different values in the Section IV-B ablation are from the independent per-configuration reruns), so it flags most genuine directional moves—the sensitivity a signal needs to be actionable. These labels describe the operating regime each configuration targets, not a validated trading strategy, whose profitability depends on the surrounding execution system beyond this work's scope. Translating this predictive accuracy into profitability is not automatic and lies outside the present scope: a mid-price-movement label is not a trading signal, because realized profit must overcome the bid–ask spread and transaction costs, survive latency-arbitrage competition, and absorb the cost of false positives that trigger unprofitable trades. Because these costs are asymmetric across classes—a missed downward move and a false upward signal carry different economic penalties—a deployed system would tune its decision thresholds to a class-specific operating point rather than maximize Macro-F1. The class-conditional recalls reported above are meant to expose this operating-point structure, not to claim a profitable strategy.

B. SCALABILITY

The folded engine is replicated by spatial unrolling, so aggregate throughput scales with on-chip logic up to the device capacity limit of Section V-G. Expressed in portable HLS with no device-specific primitives, the same source re-synthesizes on larger FPGAs, though the resource and timing figures here are specific to the KU040 and would need re-characterization on any other device [41].

C. LIMITATIONS AND FUTURE WORK

First, the accelerator is an inference core: the 57 μs figure is the on-chip, inference-only latency of a single window, and does not include feature construction, the market-data feed handler or the MAC-layer network front end. The end-to-end tick-to-trade latency of a complete trading pipeline is therefore not measured here, and integrating the core with a feed handler is left as future work [40]. In particular, the 104 handcrafted micro-structural features beyond the 40 raw levels are computed upstream by the feed handler or host CPU, which in a realistic deployment already maintains order-book state; they are not implemented in FPGA logic in this work, and estimating their on-chip resource, memory and latency cost is left as future work. The core thus realizes the inference stage, not a complete raw-market-data-to-prediction pipeline.

Second, generalization is characterized only within FI-2010. Since the pooled benchmark does not expose per-instrument identifiers, robustness is assessed across time instead: partitioning the held-out test day into five consecutive temporal strata yields INT8 Macro-F1 values with a mean of 0.826 and a standard deviation of 0.031 (range about 6.7 points), and the class balance drifts across strata (for example the upward class ranges from 45% down to 31%). The per-stratum mean (0.826) sits marginally below the whole-set 0.8331 because Macro-F1 averaged over smaller, class-shifted slices is not the pooled figure; the 0.8331 on the full day remains the valid deployment number. This quantifies the intraday non-stationarity that any deployed model must tolerate, but generalization across exchanges, instruments and shorter horizons k remains to be validated. Relatedly, the headline INT8 quantization loss of 1.78 points is that of the CF_9 deployment fold; averaged over all nine anchored folds the loss is larger—about 3.5 points, reaching roughly 10 points on the data-starved CF_5 (Section IV-C4)—so the 1.78-point figure should be read as the loss on the mature deployment fold rather than a nine-fold average, and low-bit robustness on data-starved folds is a target for future work.

Third, retargeting the accelerator to a new instrument or venue does not require redesigning the datapath, but three quantities must be recalibrated: the Z-score normalization statistics, the per-channel input and weight quantization scales and zero points (a lightweight recalibration or short QAT fine-tune), and, if the channel count changes, the folding factors so that they continue to divide the layer dimensions. All three are data-dependent parameters rather than structural changes, which is a direct benefit of the fully parameterized fixed-point datapath. However, these recalibrations adjust only deployment-time parameters and do not adapt the learned weights. Given the intraday Macro-F1 variation quantified above (mean 0.826, standard deviation 0.031), a model trained on one market period may not retain performance under a different venue or regime, so periodic fine-tuning or retraining of the model weights is likely to be necessary; how often and how much remains future work.

Fourth, the deployed LO-Arch whole-board power is now directly measured with an external DT550 in-line meter (Section VI-D); the per-configuration chip-level figures of the folding family remain Vivado post-route vectorless estimates ($\pm$10–20%), and a per-variant rail measurement of the whole folding family is left as future work.

VIII. CONCLUSION

This paper presented an end-to-end hardware–software co-design that maps a lightweight residual 1D-CNN onto a standalone edge-grade Xilinx Kintex KU040 FPGA for real-time

LOB prediction, evaluated under a leakage-controlled anchored walk-forward protocol. An INT8 QAT pipeline compresses the 86,211-parameter model by 3.38× for fully on-chip inference at a Macro-F1 of 0.8331 (a 1.78-point loss, versus 11.69 points for post-training quantization), and a Banker's-rounding fixed-point datapath is bit-exact with the PyTorch INT8 reference on all 31,828 on-board samples of the deployment-fold test day.

A controlled folding study decouples folding strategy from stream count across the design space: the fully-unrolled LO-Arch reaches a deterministic 57 μs at a directly measured 3.11 W—5.98–10.44× faster and 23.8–41.6× more energy-efficient, on an equivalent whole-device power basis, than an RTX 5070 Laptop GPU running the FP32 model at batch one (the range spanning an optimized ONNX Runtime CUDA engine and an eager-PyTorch baseline), a system-level deployment comparison rather than an equal-precision one—while folded configurations scale aggregate throughput near-linearly (demonstrated up to two streams) at lower per-engine power. The contribution is system-level: a compact INT8 1D-CNN that is synthesized, placed, routed and board-verified as a fully on-chip FPGA inference core with deterministic microsecond-scale core latency, demonstrated on an FI-2010 deployment fold rather than as a full benchmark or a complete trading system. Stronger claims such as full HFT deployment, cross-market robustness and general superiority over CPUs and GPUs are deferred to future work, which will integrate the core with a feed handler and MAC-layer front end toward an end-to-end tick-to-trade pipeline and validate generalization across exchanges, instruments and shorter horizons.

REFERENCES

[1] Z. Zhang, S. Zohren, and S. Roberts, "DeepLOB: Deep convolutional neural networks for limit order books," *IEEE Trans. Signal Process.*, vol. 67, no. 11, pp. 3001–3012, Jun. 2019.

[2] S. Yoo, H. Kim, J. Kim, S. Park, J.-Y. Kim, and J. Oh, "LightTrader: A standalone high-frequency trading system with deep learning inference accelerators and proactive scheduler," in *Proc. IEEE Int. Symp. High-Performance Comput. Archit. (HPCA)*, Montreal, QC, Canada, Feb. 2023, pp. 1017–1030.

[3] B. Wu, X. Wu, P. Li, Y. Gao, J. Si, and N. Al-Dhahir, "Efficient FPGA implementation of convolutional neural networks and long short-term memory for radar emitter signal recognition," *Sensors*, vol. 24, no. 3, p. 889, Jan. 2024.

[4] T. Ling, C. Qian, and G. Schiele, "Integer-only quantized transformers for embedded FPGA-based time-series forecasting in AIoT," in *Proc. IEEE Annu. Congr. Artif. Intell. Things (AIoT)*, Melbourne, VIC, Australia, Jul. 2024, pp. 38–44.

[5] A. Briola, S. Bartolucci, and T. Aste, "Deep limit order book forecasting: A microstructural guide," *Quant. Finance*, vol. 25, no. 7, pp. 1101–1131, Jul. 2025.

[6] M. H. Foroughi, A. Bagherzadeh, D. Rahmati, M. Foroughi, and S. Gorgin, "Financial market prediction using deep neural networks with hardware acceleration," in *Proc. 12th Int. Conf. Comput. Knowl. Eng. (ICCKE)*, Mashhad, Iran, Nov. 2022, pp. 375–381.

[7] A. Boutros, B. Grady, M. Abbas, and P. Chow, "Build fast, trade fast: FPGA-based high-frequency trading using high-level synthesis," in *Proc. Int. Conf. ReConFigurable Comput. FPGAs (ReConFig)*, Cancun, Mexico, Dec. 2017, pp. 1–6.

[8] V. Kashera, S. Jain, A. Banerjee, and S. Purini, "Building low-latency order books with hybrid binary-linear search data structures on FPGAs," in *Proc. 33rd Int. Conf. Field-Program. Log. Appl. (FPL)*, Gothenburg, Sweden, Sep. 2023, pp. 299–304.

[9] A. Ali, A. Shah, A. H. Khan, M. U. Sharif, Z. U. Zahid, R. Shahid, T. Jan, and M. H. Zafar, "Speed vs. efficiency: A framework for high-frequency trading algorithms on FPGA using Zynq SoC platform," *Alexandria Eng. J.*, vol. 96, pp. 1–14, Jun. 2024.

[10] Y. Liu, S. Chen, L. Ma, G. Yang, and K. Wan, "Real-time order book building and snapshot generating for high frequency trading on FPGA," in *Proc. IEEE 35th Int. Conf. Appl.-Specific Syst., Architectures Processors (ASAP)*, Hong Kong, Jul. 2024, pp. 71–76.

[11] R. Bosio, F. Minnella, T. Urso, M. R. Casu, L. Lavagno, M. T. Lazarescu, and P. Pasini, "NN2FPGA: Optimizing CNN inference on FPGAs with binary integer programming," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 5, pp. 1807–1818, May 2025.

[12] J. Sanchez, N. Soltani, R. Chamarthi, A. Sawant, and H. Tabkhi, "A novel 1D-convolution accelerator for low-power real-time CNN processing on the edge," in *Proc. IEEE High Perform. extreme Comput. Conf. (HPEC)*, Waltham, MA, USA, Sep. 2018, pp. 1–8.

[13] F. A. Mian and S. Zafar, "SoC-based implementation of 1-D convolutional neural network for 3-channel ECG arrhythmia classification via HLS4ML," *IEEE Embedded Syst. Lett.*, vol. 16, no. 4, pp. 429–432, Dec. 2024.

[14] P. Yun, W. Yubing, Z. Yingxu, L. Lei, Z. Jun, Z. Jianwei, Q. Li, and W. Lijun, "Hardware implementation of improved banker's fixed-point rounding algorithm," *IEEE Access*, vol. 13, pp. 36679–36686, 2025.

[15] M. Pistellato, F. Bergamasco, G. Bigaglia, A. Gasparetto, A. Albarelli, M. Boschetti, and R. Passerone, "Quantization-aware NN layers with high-throughput FPGA implementation for edge AI," *Sensors*, vol. 23, no. 10, p. 4667, May 2023.

[16] M. Klaisoongnoen, N. Brown, and O. T. Brown, "Low-power option greeks: Efficiency-driven market risk analysis using FPGAs," in *Proc. Int. Symp. Highly-Efficient Accel. Reconfigurable Technol.*, New York, NY, USA, Jun. 2022, pp. 95–101.

[17] J. Cong, B. Liu, S. Neuendorffer, J. Noguera, K. Vissers, and Z. Zhang, "High-level synthesis for FPGAs: From prototyping to deployment," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 30, no. 4, pp. 473–491, Apr. 2011.

[18] Z. Waheed, Y. Alkhrijah, S. Khalid, S. M. Riaz, S. G. Khawaja, M. U. Akram, S. M. Usman, and M. A. Alawad, "Edge-optimized CNNs: A co-designed software-hardware framework for lightweight deep learning," *IEEE Access*, vol. 13, pp. 184679–184693, 2025.

[19] S. Bertazzoni, L. Canese, G. C. Cardarilli, L. D. Nunzio, R. Fazzolari, M. Re, and S. Spanò, "Design space exploration for edge machine learning featured by MathWorks FPGA DL processor: A survey," *IEEE Access*, vol. 12, pp. 9418–9439, 2024.

[20] M. D. Gould, M. A. Porter, S. Williams, M. McDonald, D. J. Fenn, and S. D. Howison, "Limit order books," *Quant. Finance*, vol. 13, no. 11, pp. 1709–1742, 2013.

[21] A. Ntakaris, M. Magris, J. Kanniainen, M. Gabbouj, and A. Iosifidis, "Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods," *J. Forecasting*, vol. 37, no. 8, pp. 852–866, Dec. 2018.

[22] S. Kang, Y. Lee, G. Jang, and S. Lee, "Lightweight 1D-CNN-based battery state-of-charge estimation and hardware development," *Electronics*, vol. 15, no. 3, p. 704, Feb. 2026.

[23] F. Kreß, A. Serdyuk, D. Kobsar, T. Hotfilter, J. Hoefer, T. Harbaum, and J. Becker, "LOTTA: An FPGA-based low-power temporal convolutional network hardware accelerator," in *Proc. IEEE 37th Int. System-on-Chip Conf. (SOCC)*, Dresden, Germany, Sep. 2024, pp. 126–131.

[24] Z. Bao, K. Zhan, W. Zhang, and J. Guo, "LSFQ: A low precision full integer quantization for high-performance FPGA-based CNN acceleration," in *Proc. IEEE Symp. Low-Power High-Speed Chips (COOL CHIPS)*, Tokyo, Japan, Apr. 2021, pp. 1–6.

[25] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Salt Lake City, UT, USA, Jun. 2018, pp. 2704–2713.

[26] D. Gupta, A. Purohit, and R. Naresh, "FPGA for high-frequency trading: Reducing latency in financial systems," in *Proc. 3rd Int. Conf. Autom., Comput. Renew. Syst. (ICACRS)*, Pudukkottai, India, Dec. 2024, pp. 19–25.

[27] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Venice, Italy, Oct. 2017, pp. 2999–3007.

[28] M. Qasaimeh, K. Denolf, J. Lo, K. Vissers, J. Zambreno, and P. H. Jones, "Comparing energy efficiency of CPU, GPU and FPGA implementations for vision kernels," in *Proc. IEEE Int. Conf. Embedded Softw. Syst. (ICESS)*, Las Vegas, NV, USA, Jun. 2019, pp. 1–8.

[29] Y.-C. Kao, H.-A. Chen, and H.-P. Ma, "An FPGA-based high-frequency trading system for 10 gigabit Ethernet with a latency of 433 ns," in *Proc. Int. Symp. VLSI Design, Autom. Test (VLSI-DAT)*, Taiwan, Apr. 2022, pp. 1–4.

[30] R. A. Amin and R. Obermaisser, "Towards resource efficient and low latency CNN accelerator for FPGAs: Review and evaluation," in *Proc. 3rd Int. Conf. Embedded Syst. Artif. Intell. (ESAI)*, Fez, Morocco, Dec. 2024, pp. 1–10.

[31] P. Dhilleswararao, S. Boppu, M. S. Manikandan, and L. R. Cenkeramaddi, "Efficient hardware architectures for accelerating deep neural networks: Survey," *IEEE Access*, vol. 10, pp. 131788–131828, 2022.

[32] Y. Chen and K. Tanaka, "Better scalability: Improvement of block-based CNN accelerator for FPGAs," *IEEE Access*, vol. 12, pp. 187587–187603, 2024.

[33] P. Gomber and M. Haferkorn, "High-frequency-trading technologies and their implications for electronic securities trading," *Bus. Inf. Syst. Eng.*, vol. 5, no. 2, pp. 97–99, 2013.

[34] A. Tsantekidis, N. Passalis, A. Tefas, J. Kanniainen, M. Gabbouj, and A. Iosifidis, "Forecasting stock prices from the limit order book using convolutional neural networks," in *Proc. IEEE 19th Conf. Bus. Informat. (CBI)*, vol. 1, Thessaloniki, Greece, Jul. 2017, pp. 7–12.

[35] J. Woo, K. Jung, and S. Mukhopadhyay, "Efficient hardware design of DNN for RF signal modulation recognition employing ternary weights," *IEEE Access*, vol. 12, pp. 80165–80175, 2024.

[36] L. Whitford, "Multi-scale 1D convolutional networks for robust detection of anomalous financial transactions," *J. Comput. Sci. Softw. Appl.*, vol. 5, no. 7, pp. 1–10, Jul. 2025.

[37] H. Fuketa, T. Katashita, Y. Hori, and M. Hioki, "Multiplication-free lookup-based CNN accelerator using residual vector quantization and its FPGA implementation," *IEEE Access*, vol. 12, pp. 102470–102480, 2024.

[38] T. Nguyen, S. Williams, M. Siracusa, C. MacLean, D. Doerfler, and N. J. Wright, "The performance and energy efficiency potential of FPGAs in scientific computing," in *Proc. IEEE/ACM Perform. Modeling, Benchmarking Simulation High Perform. Comput. Syst. (PMBS)*, GA, GA, USA, Nov. 2020, pp. 8–19.

[39] J. W. Lockwood, A. Gupte, N. Mehta, M. Blott, T. English, and K. Vissers, "A low-latency library in FPGA hardware for high-frequency trading (HFT)," in *Proc. IEEE 20th Annu. Symp. High-Perform. Interconnects*, Santa Clara, CA, USA, Aug. 2012, pp. 9–16.

[40] R. Pottathuparambil, J. Coyne, J. Allred, W. Lynch, and V. Natoli, "Low-latency FPGA based financial data feed handler," in *Proc. IEEE 19th Annu. Int. Symp. Field-Program. Custom Comput. Mach.*, Salt Lake City, UT, USA, May 2011, pp. 93–96.

[41] Y. Wu and M. T. Arafin, "ArKANe: Accelerating Kolmogorov–Arnold networks on reconfigurable spatial architectures," *IEEE Embedded Syst. Lett.*, vol. 18, no. 1, pp. 44–47, Feb. 2026.

[42] B. Lim, S. Ö. Arik, N. Loeff, and T. Pfister, "Temporal fusion transformers for interpretable multi-horizon time series forecasting," *Int. J. Forecasting*, vol. 37, no. 4, pp. 1748–1764, Oct. 2021.

[43] D. T. Tran, A. Iosifidis, J. Kanniainen, and M. Gabbouj, "Temporal attention-augmented bilinear network for financial time-series data analysis," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 5, pp. 1407–1418, May 2019.

[44] L. Berti and G. Kasneci, "TLOB: A novel transformer model with dual attention for price trend prediction with limit order book data," 2025, *arXiv:2502.15757*.

[45] M. Prata, G. Masi, L. Berti, V. Arrigoni, A. Coletta, I. Cannistraci, S. Vyetrenko, P. Velardi, and N. Bartolini, "Lob-based deep learning models for stock price trend prediction: A benchmark study," *Artif. Intell. Rev.*, vol. 57, no. 5, p. 116, Apr. 2024.

[46] Z. Liu, C.-W. Sham, and L. Ma, "A mobile computing-friendly stock price trend prediction model," in *Proc. IEEE 13th Global Conf. Consum. Electron. (GCCE)*, Kitakyushu, Japan, Oct. 2024, pp. 210–214.

[47] R. Deng, T. Zhang, G. Gao, X. Ke, X. Xu, S. Wei, and X. Zhang, "Language-prompted visual query learning for ship-sea-land disambiguation in maritime panoptic segmentation from SAR imagery," *IEEE Trans. Circuits Syst. Video Technol.*, early access, 2026, doi: 10.1109/TCSVT.2026.3712585.

[48] R. Deng, T. Zhang, G. Gao, X. Ke, S. Wei, and X. Zhang, "Sea-state-aware conflict rectification under dual-granularity learning framework for marine SAR panoptic segmentation," *IEEE Geosci. Remote Sens. Lett.*, vol. 23, 2026, Art. no. 1504205, doi: 10.1109/LGRS.2026.3709340.

[49] X. Ke, H. Ke, S. Wei, X. Zhang, and T. Zhang, "Quad-level inshore awareness for automatic marine ship surveillance using satellite SAR system," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 19, pp. 23899–23913, 2026, doi: 10.1109/JSTARS.2026.3706505.

[50] R. Deng, T. Zhang, and G. Gao, "Language-guided semantic field query alignment for SAR ship-sea-land panoptic segmentation," *IEEE Signal Process. Lett.*, vol. 33, pp. 3406–3410, 2026, doi: 10.1109/LSP.2026.3706066.

[51] T. Zhang, X. Zhang, and G. Gao, "TLSA: A triple-level speckle awareness framework for speckle-robust marine ship surveillance using satellite synthetic aperture radar," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 62, pp. 13269–13283, 2026, doi: 10.1109/TAES.2026.3688608.

[52] T. Zhang, X. Zhang, and G. Gao, "Density knowledge mining for quantity-aware marine vessel surveillance using satellite SAR data," *IEEE Trans. Ind. Informat.*, vol. 22, no. 6, pp. 5173–5184, Jun. 2026, doi: 10.1109/TII.2026.3665231.

[53] R. Deng, T. Zhang, X. Xu, X. Zhang, and G. Gao, "Tri-state prototype self-distillation for SAR ocean imagery panoptic segmentation," *IEEE Geosci. Remote Sens. Lett.*, vol. 23, 2026, Art. no. 1503105, doi: 10.1109/LGRS.2026.3673607.

[54] T. Zhang, G. Gao, X. Ke, and X. Zhang, "Swarm learning: Perception–retrieval–localization for ship detection from synthetic aperture radar remote sensing imagery," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 19, pp. 12384–12395, 2026, doi: 10.1109/JSTARS.2026.3665805.

[55] T. Zhang, G. Gao, and X. Zhang, "Glance-focus-gaze: A novel eagle-eye vision-inspired panorama-population-individual progressive screening paradigm to capture ships in SAR images," *ISPRS J. Photogramm. Remote Sens.*, vol. 235, pp. 241–260, May 2026.

[56] Y. Xiao, C. Ventre, Y. Wang, H. Li, Y. Huan, and B. Liu, "LiT: Limit order book transformer," *Frontiers Artif. Intell.*, vol. 8, Oct. 2025, Art. no. 1616485, doi: 10.3389/frai.2025.1616485.

[57] X. Hong, C. Zheng, S. Zohren, and N. Zilberman, "Accelerating machine learning for trading using programmable switches," in *Proc. 27th Eur. Conf. Artif. Intell. (ECAI)*, 2024, pp. 1–8.



SIYUAN LIU is currently pursuing the B.E. degree in electronic information engineering with the School of Engineering, Chengdu College of University of Electronic Science and Technology of China, Chengdu, China. Her research interests include FPGA-based deep learning acceleration, hardware–software co-design, quantization-aware training for edge deployment, and real-time inference systems for financial applications.